Progetto di una

Base di Dati

Corso di Laurea in Ingegneria Informatica ed Elettronica

A.A. 2025-2026

Dipartimento di Ingegneria

Docente

Prof. Giuseppe Liotta

Piattaforma di distribuzione digitale di videogiochi

Studenti

369131 Francesca Moretti | francesca.moretti2@studenti.unipg.it

370778 Luis Alessandro Siancas Curi | luisalessandro.siancascuri@studenti.unipg.it

364408 Filippo Virgili | filippo.virgili@studenti.unipg.it

Sommario

Progettazione Concettuale....3
Raccolta dei requisiti....3
Analisi delle specifiche....5
Glossario dei termini principali....7
Elenco delle operazioni....8
Schema Entità-Relazione....9
Vincoli non esprimibili....17
Dizionario dei dati (Entità)....17
Dizionario dei dati (Relazioni)....19
Progettazione Logica....21
Tavola dei volumi....21
Tavola delle operazioni....24
Analisi delle ridondanze....26
Eliminazione delle generalizzazioni....31
Accorpamento/partizionamento di relationship....35
Scelta degli identificatori primari....36
Modello relazionale....38
Traduzione di Entità:....38
Traduzione di Relazioni:....39
Normalizzazione....40
Implementazione delle operazioni....42
Creazione delle tabelle....42
Implementazione dei vincoli....53
Creazione delle query....55
Esecuzione delle operazioni principali....87

Progettazione Concettuale

Raccolta dei requisiti

Si vuole progettare una base di dati finalizzata a modellare una piattaforma di distribuzione digitale di contenuti videoludici, tra cui videogiochi, espansioni e colonne sonore.

Per gli utenti della piattaforma si vuole rappresentare il nickname, che li identifica univocamente, l'email, la password, la nazionalità, il livello del profilo, la data di nascita, la data di iscrizione e un recapito telefonico.
La piattaforma permette all'utente di intraprendere delle relazioni di amicizia con altri utenti, si vuole riportare anche la data di inizio dell'amicizia.

Ogni utente può possedere alcune medaglie, cioè dei riconoscimenti per aver fatto qualcosa. Per ogni medaglia si riporta un codice identificativo e il nome, la stessa medaglia può appartenere a diversi utenti.
Ci sono tre tipi di medaglie: medaglie ottenute grazie all'anzianità del profilo, medaglie ottenute grazie al numero di giochi posseduti e medaglie ottenute grazie alla partecipazione ad alcuni eventi della piattaforma. Per le medaglie ottenute dal numero di giochi posseduti, si riporta anche il numero di giochi, analogamente per quelle ottenute per anzianità si segnalano gli anni di permanenza nella piattaforma, mentre se ottenuta per partecipazione ad un evento si riporta il titolo dell'evento.

La piattaforma permette agli utenti di effettuare acquisti. Per gli acquisti vogliamo tenere traccia del codice identificativo, la data in cui è stato effettuato e i dati di fatturazione, che sono rappresentati da indirizzo, CAP e città.
L'utente possiede dei metodi di pagamento con i quali può effettuare acquisti. Per un metodo di pagamento riportiamo un codice identificativo e la disponibilità. I metodi di pagamento sono due: il proprio saldo all'interno della piattaforma e un conto bancario personale, per questo si riporta l'IBAN e l'intestatario.
Si nota che un utente possiede un unico saldo, ma può registrare più di un conto bancario nel proprio profilo.
Un acquisto può essere effettuato tramite un solo metodo di pagamento, cioè l'utente non può effettuare un acquisto pagando una parte con il saldo ed un'altra con il proprio conto. Inoltre lo stesso metodo di pagamento può essere utilizzato per effettuare più acquisti.

Gli acquisti includono uno o più contenuti videoludici. Per i contenuti videoludici rappresentiamo un codice identificativo, il titolo, la descrizione, il prezzo base, il prezzo di vendita e la dimensione in GB. I contenuti videoludici sono di tre tipi: videogiochi, espansioni o colonne sonore. I contenuti devono appartenere ad almeno un genere (horror, indie, RPG, avventura…). Per il genere si rappresenta il nome, la descrizione e l'età consigliata.

Ogni utente possiede una sola libreria, composta dai contenuti acquistati. Si noti che un contenuto può essere inserito all'interno di una libreria di un utente solo se tale contenuto è stato acquistato dall'utente stesso. Per ciascun contenuto nella libreria si vogliono rappresentare l'ultima data di avvio, le ore di gioco, nel caso in cui il contenuto sia un videogioco o un espansione, e se il contenuto è tra i preferiti.

Per i videogiochi si vogliono riportare il PEGI (ossia l'età minima richiesta per il titolo), la media delle valutazioni in stelle (che varia da 1 a 5) e gli obiettivi. Si noti che un videogioco deve possedere almeno un obiettivo. Un videogioco deve necessariamente essere multiplayer o single player. Per i videogiochi si vogliono riportare le componenti hardware richieste sia minime che consigliate. Per una componente hardware si vuole riportare un identificativo, il tipo di componente e il nome di quest'ultima. Si vogliono riportare i sistemi operativi con cui i giochi sono compatibili. Per un sistema operativo riportiamo il nome e la versione che lo identificano, oltre che l'architettura e il produttore.
Un utente può lasciare una sola recensione ad ogni videogioco, solo se lo ha precedentemente acquistato. Analogamente, un contenuto può avere nessuna o più recensioni, ma solamente da utenti che lo hanno acquistato. Per ogni recensione riportiamo la valutazione in stelle da 1 a 5 e un commento opzionale.

Un'espansione fa riferimento ad un solo videogioco, tuttavia un videogioco può avere più espansioni come nessuna.

Una colonna sonora è composta da un certo numero di brani e fa riferimento ad un unico videogioco, tuttavia un videogioco può possedere come non una colonna sonora.
Per i brani si vogliono riportare il numero di traccia, il titolo e la durata. Possono esistere brani con lo stesso numero di traccia ma che sono presenti in album differenti.

La piattaforma collabora con Software House per la pubblicazione e lo sviluppo dei contenuti acquistabili che sono a disposizione dell'utente. Si vuole riportare anche la data di pubblicazione del contenuto.
Per la Software House si vuole riportare la partita iva, il nome dell'azienda e la città della sede legale. Si noti che una software house può sviluppare o pubblicare nessuno o più contenuti, oltre che occuparsi di entrambi i lavori. Un contenuto può essere pubblicato e/o sviluppato da diverse software house, inoltre una software house può sia sviluppare che pubblicare lo stesso contenuto.
Il sistema deve permettere agli utenti di seguire i profili delle Software House per ricevere aggiornamenti. Un utente può seguire diverse aziende e, specularmente, ogni azienda può avere un numero illimitato di follower. Si vuole rappresentare il seguito passato e attuale, per il seguito passato riportiamo anche la data di inizio e fine del seguito, mentre per il seguito attuale si riporta la data di inizio del seguito. Si riporta anche il numero di seguaci della Software House.

Per i videogiochi multiplayer, le software house provvedono all'amministrazione di server volti ad ospitare delle sessioni di gioco. Per una sessione di gioco si vuole riportare il codice e il numero massimo dei giocatori consentiti. Per i server vogliamo invece riportare l'indirizzo IP, il nome, lo stato del server (cioè se è aperto o chiuso) e il limite massimo di sessioni ospitabili. I server si suddividono in 2 categorie: i server proprietari, in grado di ospitare più sessioni di gioco contemporaneamente e completamente amministrati dalle software house,

ed i community server, che possono ospitare una sola sessione di gioco per volta e vengono forniti dalle software house, permettendo una parziale gestione di essi agli utenti. Si noti che più utenti possono gestire un singolo server community.

Analisi delle specifiche

Frasi di carattere generale:

Si vuole progettare una base di dati finalizzata a modellare una piattaforma di distribuzione digitale di contenuti videoludici, tra cui videogiochi, espansioni e colonne sonore.

Frasi relative agli utenti:

Per gli utenti della piattaforma si vuole rappresentare il nickname, che li identifica univocamente, l'email, la password, la nazionalità, il livello del profilo, la data di nascita, la data di iscrizione, un recapito telefonico.
La piattaforma permette all'utente di intraprendere delle relazioni di amicizia con altri utenti, si vuole riportare anche la data di inizio dell'amicizia.

Frasi relative alle medaglie:

Ogni utente può possedere alcune medaglie, cioè dei riconoscimenti per aver fatto qualcosa. Per ogni medaglia si riporta un codice identificativo e il nome, la stessa medaglia può appartenere a diversi utenti.
Ci sono tre tipi di medaglie: medaglie ottenute grazie all'anzianità del profilo, medaglie ottenute grazie al numero di giochi posseduti e medaglie ottenute grazie alla partecipazione ad alcuni eventi della piattaforma. Per le medaglie ottenute dal numero di giochi posseduti, si riporta anche il numero di giochi, analogamente per quelle ottenute per anzianità si segnalano gli anni di permanenza nella piattaforma, mentre se ottenuta per partecipazione ad un evento si riporta il titolo dell'evento.

Frasi relative agli acquisti:

La piattaforma permette agli utenti di effettuare acquisti. Per gli acquisti vogliamo tenere traccia del codice identificativo, la data in cui è stato effettuato e i dati di fatturazione, che sono rappresentati da indirizzo, CAP e città.
L'utente possiede dei metodi di pagamento con il quale può effettuare acquisti, per il metodo di pagamento riportiamo un codice identificativo e la disponibilità. I metodi di pagamento sono due: il proprio saldo all'interno della piattaforma e un conto bancario personale, per questo si riporta l'IBAN e l'intestatario.
Si nota che un utente possiede un unico saldo, ma può registrare più di un conto bancario nel proprio profilo.
Un acquisto può essere effettuato tramite un solo metodo di pagamento, cioè l'utente non può effettuare un acquisto pagando una parte con il saldo ed un'altra con il proprio conto. Inoltre lo stesso metodo di pagamento può essere utilizzato per effettuare più acquisti.

Frasi relative ai contenuti videoludici:

Gli acquisti includono uno o più contenuti videoludici. Per i contenuti videoludici rappresentiamo un codice identificativo, il titolo, la descrizione, il prezzo base e la dimensione in GB. I contenuti devono appartenere ad almeno un genere (horror, indie, RPG,

avventura…). Per il genere si rappresenta il nome, la descrizione e l'età consigliata. I contenuti videoludici sono di tre tipi: videogiochi, espansioni o colonne sonore.
Ogni utente possiede una sola libreria, composta dai contenuti acquistati. Si noti che un contenuto può essere inserito all'interno di una libreria di un utente solo se tale contenuto è stato acquistato dall'utente stesso. Per ciascun contenuto nella libreria si vogliono rappresentare l'ultima data di avvio, le ore di gioco, nel caso in cui il contenuto sia un videogioco o un espansione, e se il contenuto è tra i preferiti. Per ogni videogioco si vuole riportare la media in stelle delle valutazioni.

Frasi relative ai videogiochi:

Per i videogiochi si vogliono riportare il PEGI (ossia l'età minima richiesta per il titolo), la media delle valutazioni in stelle (che varia da 1 a 5) e gli obiettivi. Si noti che un videogioco deve possedere almeno un obiettivo. Un videogioco deve necessariamente essere multiplayer o single player. Nel caso in cui il gioco sia multiplayer, riportiamo anche il numero massimo dei giocatori. Per i videogiochi si vogliono riportare le componenti hardware richieste sia minime che consigliate. Per una componente hardware si vuole riportare un identificativo, il tipo di componente e il nome di quest'ultima. Si vogliono riportare i sistemi operativi con cui i giochi sono compatibili. Per un sistema operativo riportiamo il nome e la versione che lo identificano, oltre che l'architettura e il produttore.
Un utente può lasciare una sola recensione ad ogni videogioco, solo se lo ha precedentemente acquistato. Analogamente, un contenuto può avere nessuna o più recensioni, ma solamente da utenti che lo hanno acquistato. Per ogni recensione riportiamo la valutazione in stelle da 1 a 5 e un commento opzionale.

Frasi relative alle espansioni:

Un'espansione fa riferimento ad un solo videogioco, tuttavia un videogioco può avere più espansioni come nessuna.

Frasi relative alle colonne sonore:

Una colonna sonora è composta da un certo numero di brani e fa riferimento ad un unico videogioco, tuttavia un videogioco può possedere come non una colonna sonora.

Frasi relative ai brani:

Per i brani si vogliono riportare il numero di traccia, il titolo e la durata. Possono esistere brani con lo stesso numero di traccia ma che sono presenti in album differenti.

Frasi relative alle software house:

La piattaforma collabora con Software House per la pubblicazione e lo sviluppo dei contenuti acquistabili che sono a disposizione dell'utente. Si vuole riportare anche la data di pubblicazione del contenuto.
Per la Software House si vuole riportare la partita iva, il nome dell'azienda e la città della sede legale. Si noti che una software house può sviluppare o pubblicare nessuno o più contenuti, oltre che occuparsi di entrambi i lavori. Un contenuto può essere pubblicato e/o

sviluppato da diverse software house, inoltre una una software house può sia sviluppare che pubblicare lo stesso contenuto.
Il sistema deve permettere agli utenti di seguire i profili delle Software House per ricevere aggiornamenti. Un utente può seguire diverse aziende e, specularmente, ogni azienda può avere un numero illimitato di follower. Si vuole rappresentare il seguito passato e attuale, per il seguito passato riportiamo anche la data di inizio e fine del seguito, mentre per il seguito attuale si riporta la data di inizio del seguito. Si riporta anche il numero di seguaci della Software House.

Frasi relative ai server:
Per i videogiochi multiplayer, le software house provvedono all'amministrazione di server volti ad ospitare delle sessioni di gioco. Per una sessione di gioco si vuole riportare il codice e il numero massimo dei giocatori consentiti. Per i server vogliamo invece riportare l'indirizzo IP, il nome, lo stato del server (cioè se è aperto o chiuso) e il limite massimo di sessioni ospitabili. I server si suddividono in 2 categorie: i server proprietari, in grado di ospitare più sessioni di gioco contemporaneamente e completamente amministrati dalle software house, ed i community server, che possono ospitare una sola sessione di gioco per volta e vengono forniti dalle software house, permettendo una parziale gestione di essi agli utenti. Si noti che più utenti possono gestire un singolo server community.

Glossario dei termini principali

Termine	Descrizione	Sinonimi	Termini Collegati
Utente	Persona che si iscrive alla piattaforma e usufruisce dei suoi servizi	Profilo	Acquisto, Recensione, Medaglia, Server, Contenuto Videoludico
Medaglia	Oggetto ottenuto per aver fatto qualcosa nella piattaforma	Riconoscimento	Utente
Acquisto	Operazione effettuata dall'utente nei confronti di uno o più contenuti		Utente, Contenuto videoludico, Metodo di pagamento
Contenuto videoludico	Ciò che può essere acquistato all'interno della piattaforma	Contenuto, Contenuto acquistabile	Utente, Acquisto, Software house, Genere
Videogioco	Principale contenuto acquistabile nella piattaforma	Gioco	Hardware, Server, Contenuto videoludico, Acquisto, Utente
Espansione	Contenuto aggiuntivo ad un videogioco		Videogioco
Colonna	Raccolta musicale dei brani di	Soundtrack,	Videogioco, brano

sonora	un videogioco	Album	
Brano	Contenuto musicale all'interno del videogioco		Colonna sonora
Software house	Azienda che si occupa dello sviluppo e/o pubblicazione di un contenuto	Azienda	Contenuto videoludico, Utente, Server
Metodo di pagamento	Identifica in che modo si vuole acquistare uno o più contenuti		Acquisto, Utente, Contenuto videoludico
Server	Servizio che ospita sessioni di gioco		Videogioco, Software house, Utente
Recensione	Valutazione di un contenuto, in stelle		Contenuto videoludico, Utente

Tabella 1: glossario dei termini principali

Elenco delle operazioni

OP1: Registrazione di un utente (300 volte al mese)
OP2: Eliminazione di un utente (50 volte al mese)
OP3: Accesso di un utente (2.500 volte al giorno)
OP4: Aggiunta di un amico (2.500 volte al mese)
OP5: Rimozione di un amico (2.000 volte al mese)
OP6: Utente aumenta di livello (1.500 volte al mese)
OP7: Utente registra il proprio conto bancario (700 volte al mese)
OP8: Visualizzazione del profilo utente (300 volte al giorno)
OP9: Aggiungere all'utente una medaglia (2.500 volte al mese)
OP10: Pubblicazione di un contenuto (20 volte al mese)
OP11: Rimozione di un contenuto (5 volte al mese)
OP12: Acquisto di contenuti (200 volte al giorno)
OP13: Visualizzazione del videogioco più acquistato nel negozio (1 volta a settimana)
OP14: Utente inizia a seguire una software house (300 volte al mese)
OP15: Utente smette di seguire una software house (200 volte al mese)
OP16: Ricerca degli acquisti effettuati in una certa data da parte di un utente, segnalando la spesa totale per ciascun acquisto (3.000 volte al mese)
OP17: Ricarica del saldo di un utente (2.000 volte al mese)
OP18: Visualizzare in ordine alfabetico i contenuti all'interno della libreria di un utente (1.000 volte al giorno)
OP19: Verifica che un utente possieda l'età minima per un certo videogioco (100 volte al mese)
OP20: Visualizzazione delle espansioni di un videogioco nella libreria di un utente (500 volte al mese)
OP21: Ricerca di contenuti filtrata da prezzo, genere, data di uscita, software house editrice (8.000 volte al mese)
OP22: Visualizzazione degli obiettivi di un videogioco (500 volte al mese)
OP23: Consultazione delle specifiche hardware minime richieste per un videogioco (200 volte al mese)
OP24: Consultazione delle specifiche hardware consigliate richieste per un videogioco (200 volte al mese)
OP25: Consultazione dei sistemi operativi supportati da un videogioco (200 volte al mese)
OP26: Visualizzazione delle recensioni di un videogioco (1.100 volte al mese)
OP27: Aggiornamento del prezzo di vendita (2.000 volte al mese)
OP28: Visualizzare la valutazione media in stelle di un videogioco (1.500 volte al mese)
OP29: Aggiunta o aggiornamento di una recensione da parte di un utente (800 volte al mese)
OP30: Visualizzare le Software house che un utente segue attualmente (200 volte al mese)
OP31: Visualizzare il numero di seguaci di una Software house (200 volte al mese)
OP32: Aggiunta di un server da parte di una software house (5.000 volte al mese)
OP33: Chiusura di un server (cambiamento di stato) da parte di una software house (500 volte al mese)
OP34: Apertura di un community server (cambiamento di stato) da parte di un utente (200 volte al giorno)
OP35: Aggiunta di una software house (2 volte al mese)
OP36: Rimozione di una software house (1 volta al mese)

Schema Entità-Relazione

Per realizzare lo schema Entità-Relazione partiamo dai concetti principali della base di dati, cioè quelli di Utente, Acquisto e Contenuto Acquistabile:

- Utente identifica l'insieme delle persone registrate sulla piattaforma
- Acquisto indica l'insieme di contenuti videoludici comprati dall'utente
- Contenuto videoludico indica l'insieme di tutti i contenuti offerti dalla piattaforma, tra cui videogiochi, soundtrack ed espansioni.

Lo schema scheletro è composto dalle tre entità sopra citate e dalle relazioni che le collegano.
La relazione che lega UTENTE e ACQUISTO è Transazione, le sue occorrenze rappresentano gli acquisti effettuati dall'utente sulla piattaforma.
La relazione che lega ACQUISTO e CONTENUTO VIDEOLUDICO è Composizione, le sue occorrenze rappresentano l'insieme dei contenuti presenti in un determinato acquisto.



Figura 1: Schema scheletro (primo livello)

A questo punto si può espandere lo schema aggiungendo la relazione ricorsiva Amicizia, che permette di rappresentare il fatto che l'utente possa avere amici sulla piattaforma.

Per indicare che l'utente può ottenere delle medaglie, aggiungiamo l'entità MEDAGLIA e la colleghiamo a UTENTE tramite la relazione Merito, le cui occorrenze rappresentano le medaglie che l'utente possiede sul profilo. Si vuole distinguere tra diversi tipi di medaglie perciò suddividiamo, tramite una generalizzazione totale ed esclusiva, l'entità MEDAGLIA in ANZIANITÀ, GIOCO, EVENTO.

Analogamente, per l'entità CONTENUTO, vogliamo distinguere i diversi tipi di contenuti, quindi suddividiamo l'entità in COLONNA SONORA, ESPANSIONE e VIDEOGIOCO tramite una generalizzazione totale ed esclusiva.

Per i videogiochi vogliamo distinguere se questi sono MULTIPLAYER o SINGLEPLAYER, per cui creiamo una generalizzazione totale ed esclusiva.
Lo schema ottenuto a questo punto è il seguente:

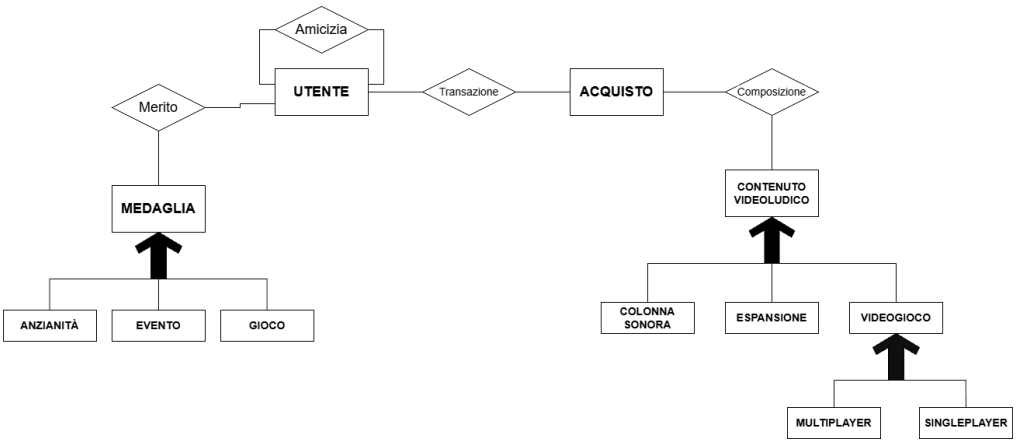


Figura 2: Evoluzione schema scheletro (secondo livello)

Per raffinare ulteriormente lo schema aggiungiamo l'entità METODO DI PAGAMENTO, collegato allo schema tramite la relazione Possesso con UTENTE e la relazione Tramite con ACQUISTO, che indicano che l'utente possiede dei metodi di pagamento e con essi può effettuare acquisti. Vogliamo distinguere tra due metodi di pagamento, perciò creiamo una generalizzazione totale ed esclusiva per METODO DI PAGAMENTO che genera le entità SALDO e CONTO BANCARIO.

Inoltre, si vuole riportare l'appartenenza di un contenuto videoludico ad un genere, si creano quindi l'entità GENERE e la relazione Appartenenza che la lega all'entità CONTENUTO VIDEOLUDICO.

Si vogliono ora espandere le informazioni che riguardano le entità COLONNA SONORA, ESPANSIONE e VIDEOGIOCO.
Per indicare che ogni espansione fa riferimento ad un videogioco creiamo una relazione chiamata Estensione Di che collega ESPANSIONE a VIDEOGIOCO.
Inoltre per indicare che una colonna sonora fa riferimento ad un videogioco creiamo la relazione Relativa A che collega COLONNA SONORA a VIDEOGIOCO.
Si vuole rappresentare che le colonne sonore sono composte da diversi brani, aggiungiamo quindi la relazione Componimento che lega BRANO a COLONNA SONORA.

Per quanto riguarda le recensioni, basta aggiungere una relazione chiamata Recensione che collega UTENTE a VIDEOGIOCO.

Si vuole esprimere il fatto che un utente possiede una libreria composta da contenuti, perciò si lega l'entità UTENTE a CONTENUTO VIDEOLUDICO tramite la relazione Libreria.

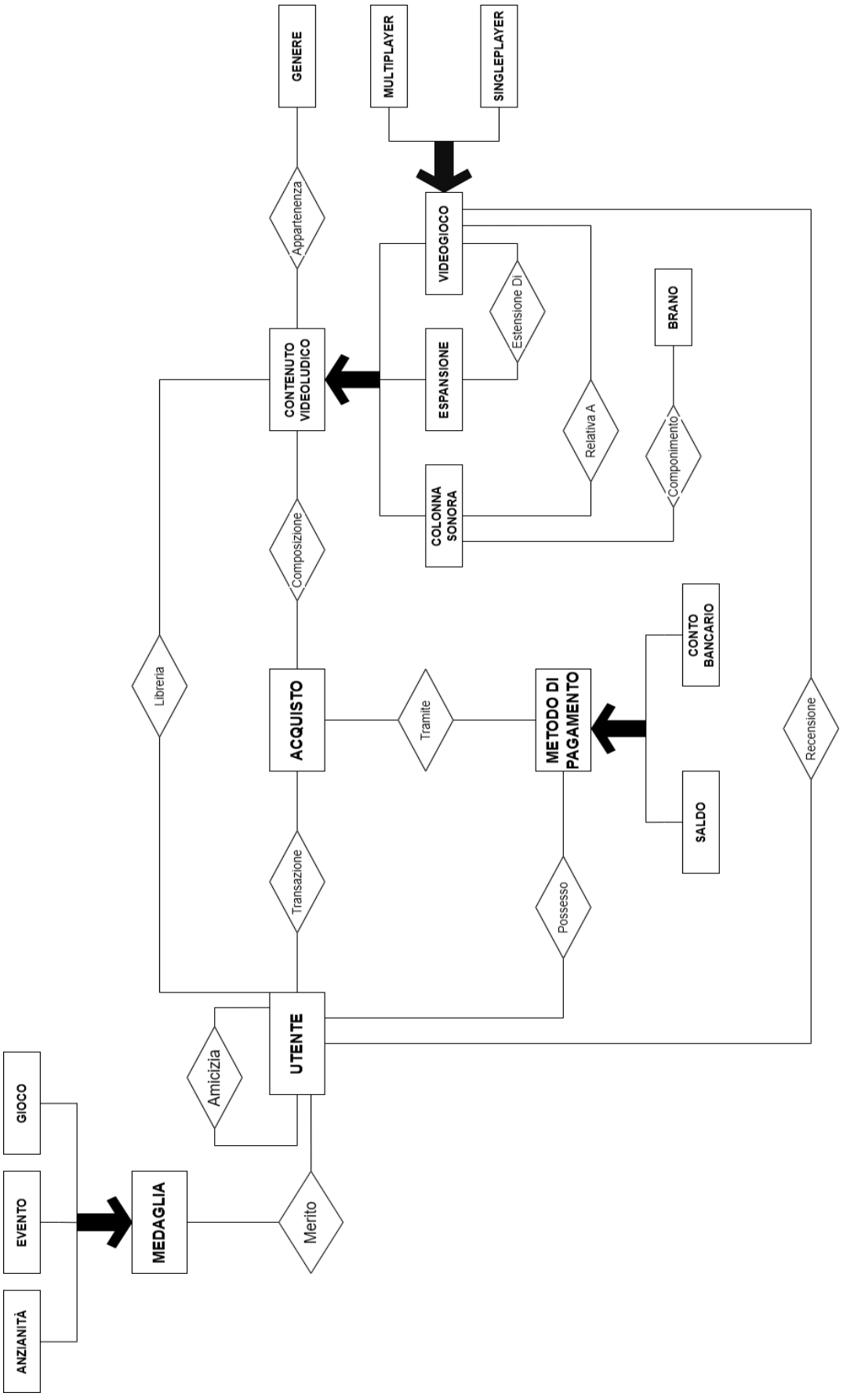


Figura 3: Evoluzione schema (terzo livello)

Mancano le informazioni che riguardano le software house, creiamo l'entità SOFTWARE HOUSE e la leghiamo a CONTENUTO VIDEOLUDICO tramite le relazioni Pubblicazione e Sviluppo, che indicano rispettivamente la pubblicazione e lo sviluppo di contenuti acquistabili da parte delle aziende.

Si vuole riportare il fatto che l'utente possa seguire le Software House, quindi si creano due relazioni che rappresentano il Seguito Passato e il Seguito Attuale tra UTENTE e SOFTWARE HOUSE.

Si espande ulteriormente la relazione VIDEOGIOCO, esprimendo i requisiti minimi e consigliati tramite le relazioni Consigliato e Minimo, che legano VIDEOGIOCO all'entità HARDWARE. Rappresentiamo anche il fatto che i videogiochi sono compatibili solo con alcuni sistemi operativi, perciò creiamo l'entità SISTEMA OPERATIVO, legata a VIDEOGIOCO tramite la relazione Compatibilità.

Come ultima cosa si vogliono aggiungere le informazioni che riguardano i server, la loro amministrazione e utilizzo. Creiamo quindi l'entità SERVER e rappresentiamo attraverso la relazione Amministrazione che lega SERVER a SOFTWARE HOUSE, che i server sono gestiti da queste ultime.
I server hanno il compito di ospitare sessioni di gioco, le quali sono relative ai soli giochi multiplayer, perciò si crea l'entità SESSIONE e si lega a SERVER con la relazione Ospitata e a MULTIPLAYER con la relazione Connessione.

Vogliamo distinguere tra due tipi di server: tramite una generalizzazione totale ed esclusiva si creano due entità PROPRIETARIO e COMMUNITY. Inoltre sappiamo che gli utenti possono gestire solamente i server community, perciò si aggiunge la relazione Gestione tra UTENTE e COMMUNITY.

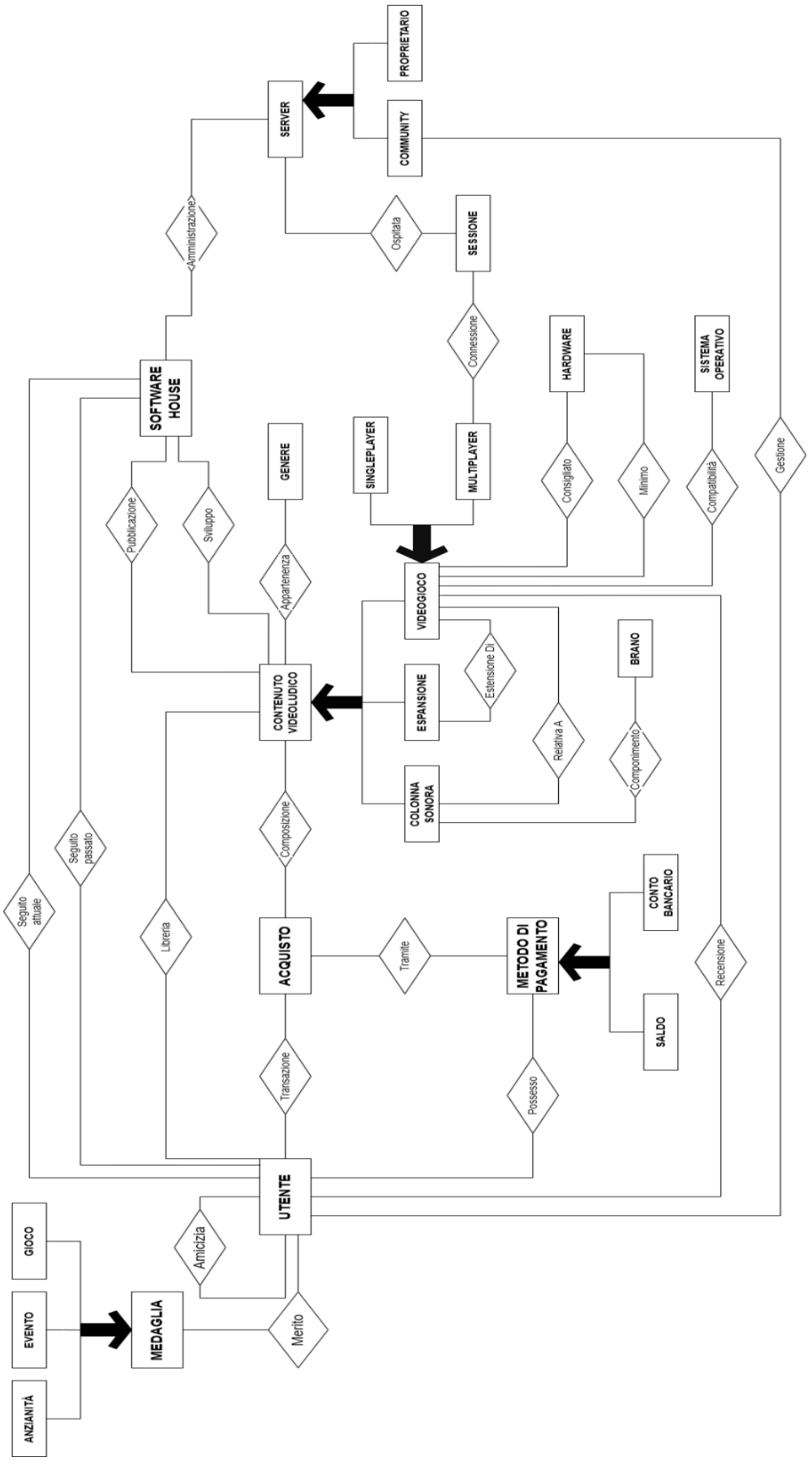


Figura 4: Evoluzione schema (quarto livello)

Come ultima cosa, completiamo lo schema in Figura 4 aggiungendo ulteriori informazioni tra cui gli attributi e le cardinalità delle relazioni. Gli attributi aggiunti sono ricavati dall'analisi delle specifiche.

Lo schema completo è il seguente:

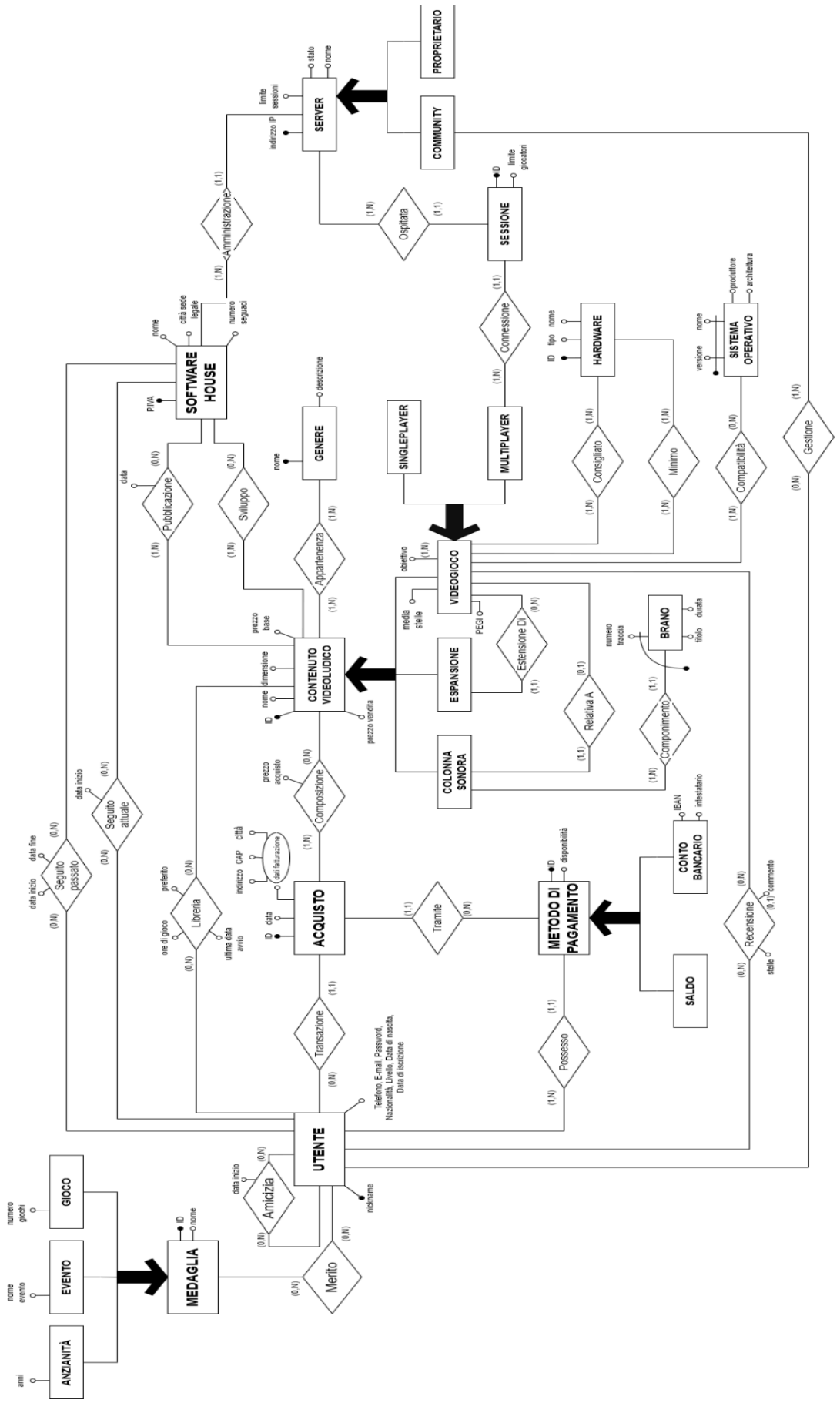


Figura 5: Schema completo

Vincoli non esprimibili

- Una recensione può essere inserita dall'utente solo se questo ha precedentemente acquistato il videogioco.
- L'utente può pubblicare una sola recensione per gioco.
- Il contenuto si trova in libreria utente solo se è stato precedentemente acquistato dallo stesso utente.
- Un utente possiede un solo saldo nella piattaforma, ma può collegare più conti bancari
- Un server di tipo community può ospitare una sola sessione.

Dizionario dei dati (Entità)

Entità	Identificatore	Attributi	Descrizione
UTENTE	Nickname	Telefono, E-mail, Password, Nazionalità, Livello, Data di nascita, Data di iscrizione	Dati relativi agli utenti iscritti alla piattaforma
MEDAGLIA	ID medaglia	Nome	Nomi dei premi acquisibili
ANZIANITÀ	ID medaglia	Anni	Premi acquisiti per data di iscrizione sulla piattaforma
GIOCO	ID medaglia	Numero giochi	Premi acquisiti per il numero di videogiochi acquistati sulla piattaforma
EVENTO	ID medaglia	Nome evento	Premi acquisiti per la partecipazione ad eventi della piattaforma
ACQUISTO	ID acquisto	Data, Dati di fatturazione	Dati relativi agli acquisti eseguiti sulla piattaforma

METODO DI PAGAMENTO	ID	Disponibilità	Rappresenta i metodi di pagamento e i relativi bilanci
SALDO	ID		Portafogli personali presenti sulla piattaforma
CONTO BANCARIO	ID	IBAN, Intestatario	Rappresenta i dati bancari registrati dagli utenti per prelevare i fondi esternamente
CONTENUTO VIDEOLUDICO	ID contenuto	Nome, Dimensione, Prezzo di vendita, Prezzo base	Dati dei prodotti messi in vendita dalla piattaforma
GENERE	Nome	Descrizione	Classificazione in categoria di un videogioco
VIDEOGIOCO	ID contenuto	Obiettivo, Media stelle, PEGI	Rappresenta il principale prodotto acquistabile
MULTIPLAYER	ID contenuto		Modalità di gioco che permette a più utenti di giocare insieme
SINGLE PLAYER	ID contenuto		Modalità gioco individuale
ESPANSIONE	ID contenuto		Contenuto aggiuntivo che estende il videogioco
COLONNA SONORA	ID contenuto		Raccolte musicali dei brani appartenenti ad un videogioco
BRANO	Numero di traccia, ID contenuto_Colonna sonora	Titolo, Durata	Singoli contenuti musicali che appartengono ad una raccolta
HARDWARE	ID hardware	Tipo, Nome	Dati delle componenti fisiche dei computer

SISTEMA OPERATIVO	Nome, Versione	Produttore, Architettura	Dati relativi ai sistemi operativi dei computer
SOFTWARE HOUSE	P.IVA	Nome, Città Sede Legale, Numero Seguaci	Dati relativi a un'azienda videoludica che sviluppa e/o pubblica contenuti
SERVER	Indirizzo IP	Stato, Limite sessioni, Nome	Dettagli relativi ai server
SESSIONE	ID sessione	Limite giocatori	Sessioni di gioco alle quali l'utente può partecipare
PROPRIETARIO	Indirizzo IP		Server amministrati direttamente dalle software house
COMMUNITY	Indirizzo IP		Server delle software house gestiti parzialmente dagli utenti

Tabella 2: Dizionario dei dati (entità)

Dizionario dei dati (Relazioni)

Relazione	Entità	Attributi	Descrizione
AMICIZIA	Utente, Utente	DataInizio	Relazione stabilita tra due utenti
TRANSAZIONE	Utente, Acquisto		Mostra tutti gli acquisti effettuati da un utente
COMPOSIZIONE	Acquisto, Contenuto Videoludico	Prezzo acquisto	Mostra i contenuti compresi in un acquisto
MERITO	Utente, Medaglia		Mostra le medaglie conseguite da un utente
POSSESSO	Utente, Metodo di pagamento		Mostra i metodi di pagamento posseduti dall'utente

TRAMITE	Acquisto, Metodo di pagamento		Segnaappartenenza la il metodo di pagamento usato per un acquisto
APPARTENENZA	Contenuto videoludico, Genere		Mostra il genere di un contenuto
ESTENSIONE DI	Espansione, Videogioco		Mostra a il videogioco a cui è relativa un'espansione
RELATIVA A	Colonna sonora, Videogioco		Mostra la colonna sonora relativa a un videogioco
COMPONIMENTO	Colonna sonora, Brano		Mostra i brani contenuti in una colonna sonora
RECENSIONE	Utente, Videogioco	Stelle, Commento	Mostra i videogiochi e i rispettivi utenti che li hanno recensiti
LIBRERIA	Utente, Contenuto Videoludico	UltimaDataAvvio, OreDiGioco, Preferiti	Mostra l'insieme dei contenuti posseduti dall'utente
MINIMO	Videogioco, Hardware		Mostra le componenti hardware minime richieste per un videogioco
CONSIGLIATO	Videogioco, Hardware		Mostra le componenti hardware consigliate per un videogioco
COMPATIBILITÀ	Videogioco, Sistema operativo		Mostra i sistemi operativi compatibili con un dato videogioco
SVILUPPO	Software house, Contenuto videoludico		Mostra le software house che hanno contribuito allo sviluppo di un videogioco
PUBBLICAZIONE	Software house, Contenuto videoludico	DataPubblicazione	Mostra le software house che si sono occupate della

			pubblicazione di un videogioco
SEGUITO ATTUALE	Utente, Software house	DataInizio	Mostra gli utenti seguaci attuali di una software house
SEGUITO PASSATO	Utente, Software house	DataInizio, DataFine	Mostra gli utenti seguaci passati di una software house
CONNESSIONE	Multiplayer, Sessione		Mostra le sessioni online di un videogioco multiplayer
OSPITATA	Sessione, Server		Mostra i server che ospitano le sessioni di gioco
GESTIONE	Community, Utente		Mostra gli utenti che gestiscono un dato community server
AMMINISTRAZIONE	Software House, Server		Mostra le software house che si occupano dell'amministrazione dei relativi server

Tabella 3: Dizionario dei dati (relazioni)

Progettazione Logica

Tavola dei volumi

Concetto	Tipo	Volume
UTENTE	E	10.000
MEDAGLIA	E	50
ANZIANITÀ	E	10
GIOCO	E	15
EVENTO	E	25
ACQUISTO	E	5 (acquisti/utente) x 10.000 (utenti) = 50.000
METODO DI PAGAMENTO	E	15.000
SALDO	E	1 (saldo/utente) x 10.000 (utenti) = 10.000
CONTO BANCARIO	E	0.5 (conti/utente) x 10.000 (utenti) = 5.000
CONTENUTO VIDEOLUDICO	E	1.000
GENERE	E	20
VIDEOGIOCO	E	500
MULTIPLAYER	E	200
SINGLE PLAYER	E	300
ESPANSIONE	E	0.6 (espansioni/videogioco) x 500 (videogiochi) = 300
COLONNA SONORA	E	0.4 (colonne sonore/videogioco) x 500 (videogiochi) = 200
BRANO	E	12 (brani/colonna sonora) x 200 (colonne sonore) = 2.400
HARDWARE	E	2200
SISTEMA OPERATIVO	E	15
SOFTWARE HOUSE	E	100
SERVER	E	800
PROPRIETARIO	E	200
COMMUNITY	E	600
SESSIONE	E	7 (sessioni/server proprietario) x 200 (server proprietari) + 1 (sessione/server

		community) x 600 (server community) = 2.000
AMICIZIA	R	2.5 (amicizie/utente) x 10.000 (utenti) x 2 = 50.000
TRANSAZIONE	R	50.000
COMPOSIZIONE	R	0.75+0.45+0.3 ((videogiochi, espansioni, colonne sonore)/acquisto) x 50.000 acquisti = 1.5 (contenuti videoludici/acquisto) x 50.000 acquisti = 75.000
MERITO	R	3 (medaglie/utente) x 10.000 (utenti) = 30.000
POSSESSO	R	15.000
TRAMITE	R	50.000
APPARTENENZA	R	2.5 (generi/contenuto videoludico) x 1.000 (contenuti videoludici) = 2.500
ESTENSIONE DI	R	300
RELATIVA A	R	200
COMPONIMENTO	R	2.400
RECENSIONE	R	37.500 (videogiochi installati) x 0.2 (tasso di recensione) = 7.500
LIBRERIA	R	1.5 (contenuti installati/acquisto) x 5 (acquisti/utente) x 10.000 (utenti) = 75.000
MINIMO	R	4 (requisiti hardware/videogioco) x 500 (videogiochi) = 2.000
CONSIGLIATO	R	4 (requisiti hardware/videogioco) x 500 (videogiochi) = 2.000
COMPATIBILITÀ	R	2.5 (sistemi operativi/videogioco) x 500 (videogiochi) = 1.250
SVILUPPO	R	1.5 (software house developer/contenuto videoludico) x 1.000 (contenuti videoludici) = 1.500
PUBBLICAZIONE	R	1.5 (software house publisher/contenuto videoludico) x 1.000 (contenuti videoludici) = 1.500
SEGUITO ATTUALE	R	0.5 (software house/utente) x 10.000 (utenti) = 5.000

SEGUITO PASSATO	R	1 (software house/utente) x 10.000 (utenti) = 10.000
CONNESSIONE	R	2.000
OSPITATA	R	2.000
GESTIONE	R	2 (utenti/server community) x 600 (server community) = 1.200
AMMINISTRAZIONE	R	800

Tabella 4: Tavola dei volumi

Tavola delle operazioni

Operazione	Tipo	Frequenza
OP1: Registrazione di un utente	I	300 volte al mese
OP2: Eliminazione di un utente	I	50 volte al mese
OP3: Accesso di un utente	I	2.500 volte al giorno
OP4: Aggiunta di un amico	I	2.500 volte al mese
OP5: Rimozione di un amico	I	2.000 volte al mese
OP6: Utente aumenta di livello	I	1.500 volte al mese
OP7: Utente registra il proprio conto bancario	I	700 volte al mese
OP8: Visualizzazione del profilo utente	I	300 volte al giorno
OP9: Aggiungere all'utente una medaglia	B	2.500 volte al mese
OP10: Pubblicazione di un contenuto	B	20 volte al mese
OP11: Rimozione di un contenuto	B	5 volte al mese
OP12: Acquisto di contenuti	I	200 volte al giorno
OP13: Visualizzazione del videogioco più acquistato nel negozio	B	1 volta a settimana

OP14: Utente inizia a seguire una Software House	I	300 volte al mese
OP15: Utente smette di seguire una Software House	I	250 al mese
OP16: Ricerca degli acquisti effettuati in una certa data da parte di un utente, segnalando la spesa totale per ciascun acquisto	I	3.000 al mese
OP17: Ricarica del saldo di un utente	I	2.000 volte al mese
OP18: Visualizzare in ordine alfabetico i contenuti nella libreria di un utente	I	1.000 volte al giorno
OP19: Verifica che un utente possieda l'età minima per un certo videogioco	I	100 volte al mese
OP20: Visualizzazione delle espansioni di un videogioco	I	500 volte al mese
OP21: Ricerca di contenuti filtrata da prezzo, genere, data di uscita, software house editrice	I	8.000 volte al mese
OP22: Visualizzazione degli obiettivi di un videogioco	I	500 volte al mese
OP23: Consultazione delle specifiche hardware minime richieste per un videogioco	I	200 volte volte al mese
OP24: Consultazione delle specifiche hardware consigliate richieste per un videogioco	I	200 volte al mese
OP25: Consultazione dei sistemi operativi supportati da un videogioco	I	200 volte al mese
OP26: Visualizzazione delle recensioni di un videogioco	I	1.100 volte al mese
OP27: Aggiornamento del prezzo di vendita	I	2.000 volte al mese
OP28: Visualizzare la valutazione media in stelle di un videogioco	I	1.500 volte al mese

OP29: Aggiunta o aggiornamento di una recensione da parte di un utente	I	800 al mese
OP30: Visualizzare le Software house che un utente segue attualmente	I	200 al mese
OP31: Visualizzare il numero di seguaci di una Software house	I	200 al mese
OP32: Aggiunta di un server da parte di una software house	I	5.000 al mese
OP33: Chiusura di un server (cambiamento di stato) da parte di una software house	I	500 al giorno
OP34: Apertura di un community server (cambiamento di stato) da parte di un utente	I	200 al giorno
OP35: Aggiunta di una software house	B	2 volte al mese
OP36: Rimozione di una software house	B	1 volta al mese

Tabella 5: Tavola delle operazioni

Analisi delle ridondanze

Una ridondanza, in uno schema concettuale, consiste nella presenza di un dato che può essere derivato da altri dati mediante una serie di operazioni. L'analisi delle ridondanze, basata sui costi in termini di spazio e tempo, è necessaria per comprendere se una data ridondanza comporta nel complesso un vantaggio od uno svantaggio.

Una possibile ridondanza è quella che riguarda il numero di seguaci di una software house: tale numero può infatti essere ottenuto conteggiando le occorrenze della relazione Seguito Attuale.

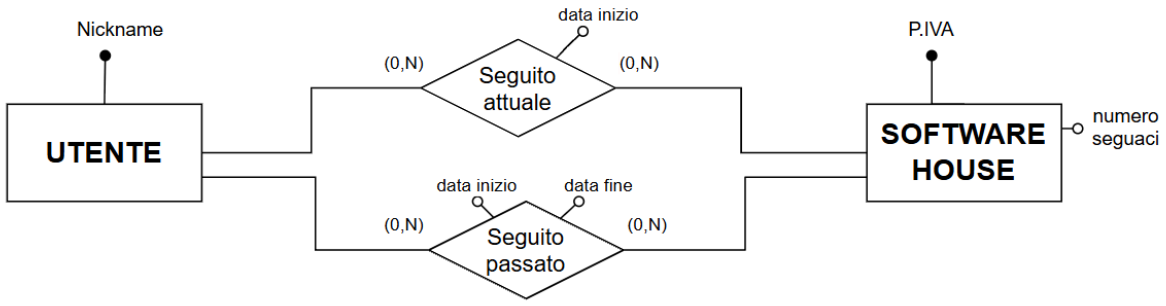


Figura 6: Schema con ridondanza

Per stabilire se conviene mantenere la ridondanza o meno, si analizzano gli indici di prestazione in presenza e in assenza di ridondanza. Si calcola l'indice di prestazione sulla base delle operazioni che coinvolgono le relazioni ed entità oggetto di ridondanza.
In questo caso le operazioni da considerare sono:

OP14: Utente inizia a seguire una software house (300 volte al mese)

OP15: Utente smette di seguire una software house (200 volte al mese)

OP31: Visualizzare il numero di seguaci di una Software house (200 volte al mese)

Si valutano ora le prestazioni in presenza di ridondanza, quindi considerando la presenza dell'attributo Numero seguaci.

Tavola degli accessi per **OP14**:

Concetto	**Costrutto**	**Accessi**
Utente	Entità	1 Lettura
Seguito attuale	Relazione	1 Scrittura
Software House	Entità	1 Lettura
Software House	Entità	1 Scrittura

Tavola degli accessi per **OP15**:

Concetto	**Costrutto**	**Accessi**
Utente	Entità	1 Lettura
Software House	Entità	1 Lettura
Seguito attuale	Relazione	1 Scrittura
Seguito passato	Relazione	1 Scrittura
Software House	Entità	1 Scrittura

Tavola degli accessi per **OP31**:

Concetto	**Costrutto**	**Accessi**
Software House	Entità	1 Lettura

Considerando il peso di una scrittura pari a 2 letture, si moltiplicano i risultati parziali per la frequenza delle operazioni e si ottengono i costi totali al mese in presenza di ridondanza:

$$(2L+2S) \times 300 + (2L+3S) \times 200 + (1L) \times 200 =$$

6L x 300 + 8L x 200 + 1L x 200 = 1.800L + 1.600L + 200L = 3.600 letture al mese

Dalla tavola dei volumi si ha che 5000 utenti, dei 10.000 nella piattaforma, seguono le Software House. Sempre dalla tavola dei volumi emerge che queste sono 100, per cui si hanno in media 50 utenti seguaci di una Software house. E' quindi necessario 1 byte per rappresentare l'attributo ridondante.
Moltiplicando la dimensione dell'attributo per il numero di occorrenze dell'entità SOFTWARE HOUSE otteniamo lo spreco di memoria:

1 (byte) x 100 (Software House) = 100 byte

Si valutano ora i costi in assenza di ridondanza, lo schema è il seguente:

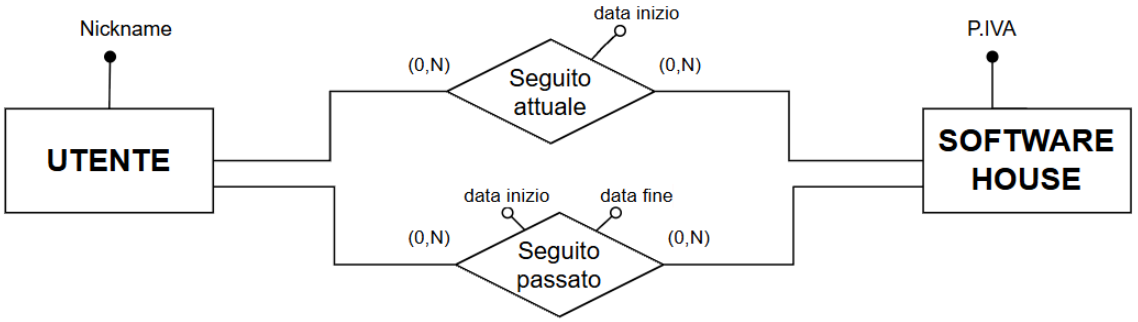


Figura 7: Schema senza ridondanza

Tavola degli accessi per **OP14**:

Concetto	**Costrutto**	**Accessi**
Utente	Entità	1 Lettura
Seguito attuale	Relazione	1 Scrittura

Tavola degli accessi per **OP15**:

Concetto	**Costrutto**	**Accessi**
Utente	Entità	1 Lettura
Seguito attuale	Relazione	1 Scrittura
Seguito passato	Relazione	1 Scrittura

Tavola degli accessi per **OP31**:

Concetto	**Costrutto**	**Accessi**
Software House	Entità	1 Lettura
Seguito attuale	Relazione	50 Letture

Si calcolano i costi totali senza ridondanza:

(1L+1S) x 300 + (1L+2S) x 200 + (51L) x 200 =

3L x 300 + 5L x 200 + 51L x 200 = 900L + 1.000L + 10.200L = 12.100 letture al mese

Risulta che gli accessi con ridondanza sono minori di quelli senza, perciò si sceglie di mantenere la ridondanza al netto dello spreco di memoria.

Un'altra possibile ridondanza è quella che riguarda la media stelle di un videogioco, in quanto può essere ottenuta sommando tutte le recensioni in stelle e dividendo per il numero totale di recensioni del rispettivo videogioco.

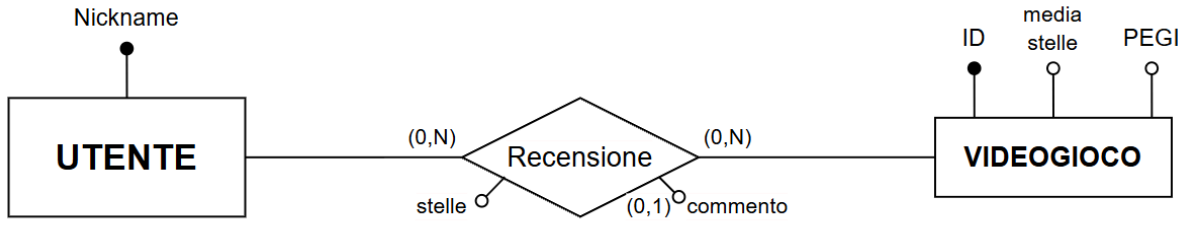


Figura 8: Schema con ridondanza

Analogamente a prima, per stabilire se conviene mantenere la ridondanza, si analizzano gli accessi sulla base delle operazioni che coinvolgono l'entità oggetto di ridondanza.
Le operazioni in questione sono:

OP28: Visualizzare la valutazione media in stelle di un videogioco (1.500 volte al mese)

OP29: Aggiunta o aggiornamento di una recensione da parte di un utente (800 volte al mese)

Valutiamo i costi in presenza di ridondanza, quindi con l'attributo Media stelle sull'entità VIDEOGIOCO.

Tavola degli accessi per **OP28**:

Concetto	**Costrutto**	**Accessi**
Videogioco	Entità	1 Lettura

Tavola degli accessi per **OP29**:

Concetto	**Costrutto**	**Accessi**
Utente	Entità	1 Lettura

Recensione	Relazione	1 Scrittura
Videogioco	Entità	1 Lettura
Videogioco	Entità	1 Scrittura

Si calcolano i costi totali mantenendo la ridondanza:

1L x 1.500 + (2L+2S) x 800 = 1L x 1.500 + 6L x 800 = 1.500L + 4.800L =

= 6.300 letture al mese

Si calcola ora lo spazio occupato dall'attributo ridondante nel database: le recensioni oscillano da 1.0 a 5.0 stelle, considerando che stiamo lavorando con numeri decimali, usiamo 2 byte per ogni videogioco. Dalla tavola dei volumi risultano 500 videogiochi presenti nel catalogo, per cui lo spreco di memoria corrisponde a:

2 (byte) x 500 (videogiochi) = 1.000 byte ≃ 1 Kbyte

Si valutano ora i costi in assenza di ridondanza, lo schema è il seguente:

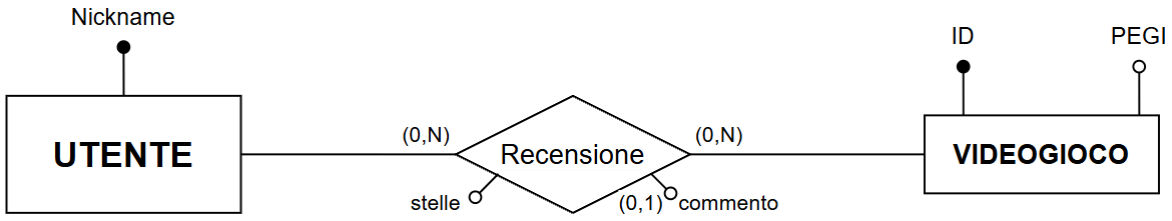


Figura 9: Schema senza ridondanza

Tavola degli accessi per **OP28**:

Concetto	**Costrutto**	**Accessi**
Videogioco	Entità	1 Lettura
Recensione	Relazione	15 Letture

(dati 500 videogiochi disponibili nel catalogo e un totale di 7.500 recensioni, si ricava che un videogioco ha in media 15 recensioni)

Tavola degli accessi per **OP29**:

Concetto	**Costrutto**	**Accessi**
Utente	Entità	1 Lettura
Recensione	Relazione	1 Scrittura

Si calcolano i costi totali senza ridondanza:

16L x 1.500 + (1L+1S) x 800 = 16L x 1.500 + 3L x 800 =

= 24.000L + 2.400 = 26.400 letture al mese

Risulta evidente come sia più conveniente mantenere la ridondanza, al netto dello spreco di circa 1Kbyte di memoria nel database.

Si osservano infine gli attributi Prezzo vendita e Prezzo acquisto, rispettivamente di CONTENUTO VIDEOLUDICO e Composizione. Nonostante i nomi possano alludere ad una ridondanza, è necessario specificare che il significato delle due espressioni presenta nette differenze. Prezzo vendita non è altro che il prezzo di un contenuto nel momento in cui esso viene visualizzato nel negozio, il che significa che l'occorrenza relativa a un contenuto viene sovrascritta nell'esatto momento in cui questa cambia prezzo di vendita. L'attributo Prezzo acquisto vuole invece segnalare la spesa eseguita per un certo contenuto nella data in cui esso è stato acquistato, ha quindi valenza storica, e l'occorrenza che relaziona ACQUISTO con CONTENUTO VIDEOLUDICO non verrà mai sovrascritta.
Ne consegue che la rimozione di Prezzo acquisto impedirebbe poi di capire quanto un contenuto è stato pagato in una certa data (e conseguentemente sarebbe impossibile calcolare la spesa totale di un acquisto), mentre la rimozione di Prezzo vendita impedirebbe invece di osservare il prezzo attuale di un contenuto, in quanto si potrebbe solo fare affidamento al prezzo di acquisto di tale contenuto nella data più recente in cui è stato acquistato (e chiaramente il prezzo di vendita potrebbe cambiare tra il momento attuale e quello dell'acquisto passato, ma non sarebbe visibile).
Di conseguenza, si mantengono entrambi gli attributi, di seguito lo schema relativo:

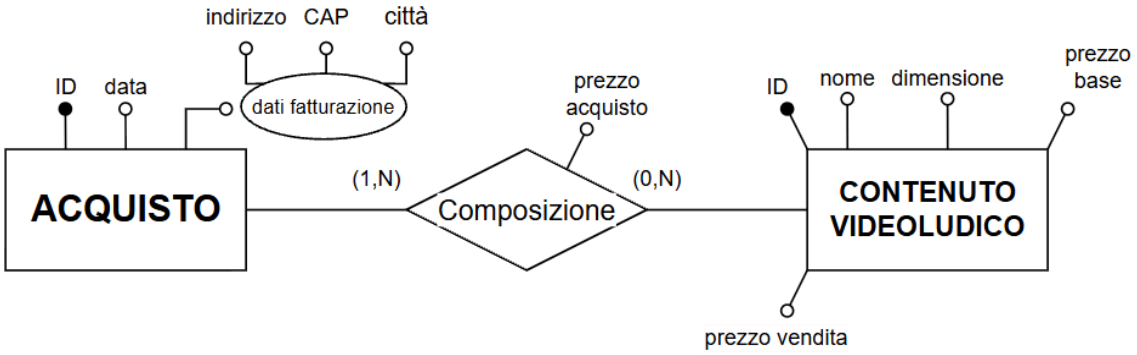


Figura 10: analisi dell'attributo Prezzo di acquisto

Eliminazione delle generalizzazioni

Le generalizzazioni non sono direttamente rappresentabili nei tradizionali sistemi per la gestione delle basi di dati. Per questo motivo, in questa fase, si cerca di trasformare tali costrutti tramite l'utilizzo esclusivo di entità e relazioni.

Lo schema finale ottenuto presenta diverse generalizzazioni, partiamo con quella che riguarda le MEDAGLIE dell'utente. Si effettua un collassamento verso l'alto in quanto nelle operazioni non si fa alcuna distinzione tra le occorrenze delle entità figlie o dell'entità padre. Nella Figura 10 si rappresenta lo schema prima e dopo l'analisi.

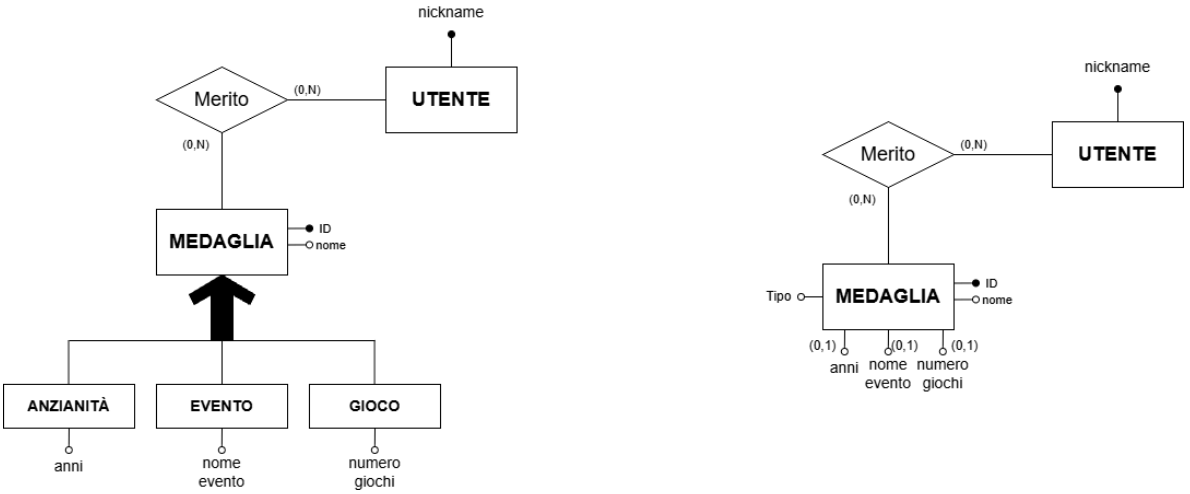


Figura 11: Analisi della generalizzazione MEDAGLIA

Un'altra generalizzazione riguarda i METODI DI PAGAMENTO, in questo caso le operazioni riguardano principalmente le entità figlie. Tuttavia, se volessimo effettuare un collassamento verso il basso, dovremmo collegare le entità figlie ad ACQUISTO tramite due relazioni distinte. Conseguentemente, ogni volta che si registra un acquisto, il database deve capire in quale delle due tabelle si trova il metodo di pagamento utilizzato.
Analogamente, dovremmo legare le entità figlie SALDO e CONTO BANCARIO anche a UTENTE, sempre utilizzando due relazioni. Perciò, per semplicità, collassiamo l'entità verso l'alto.
La seguente figura mostra l'analisi appena fatta.

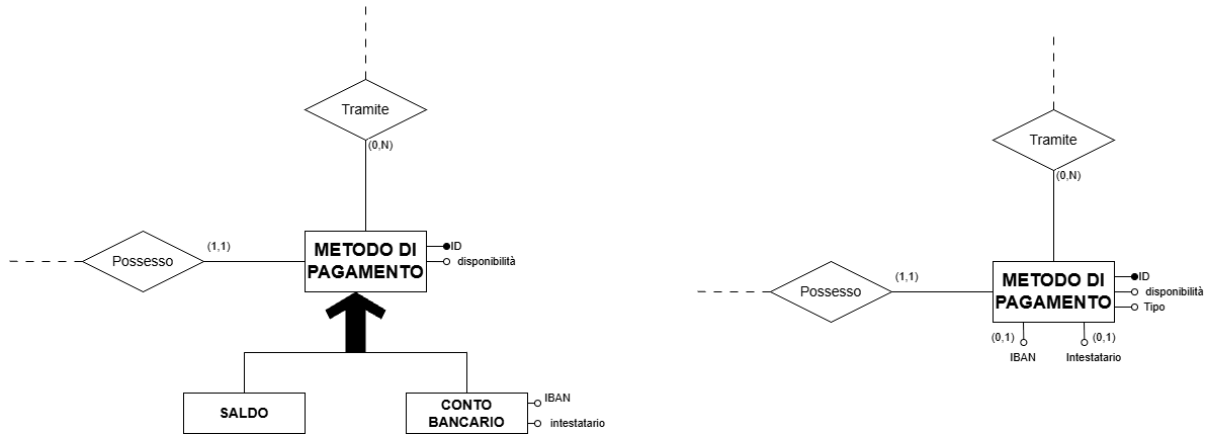


Figura 12: Analisi della generalizzazione METODO DI PAGAMENTO

Procediamo analizzando la generalizzazione principale, ovvero quella di CONTENUTO VIDEOLUDICO, procedendo prima con l'analisi della sotto generalizzazione VIDEOGIOCO. Nelle operazioni non si fa mai distinzione tra videogiochi multiplayer o single player, per questo basta accorpare le figlie nel genitore.
Di seguito si mostra l'analisi della generalizzazione.

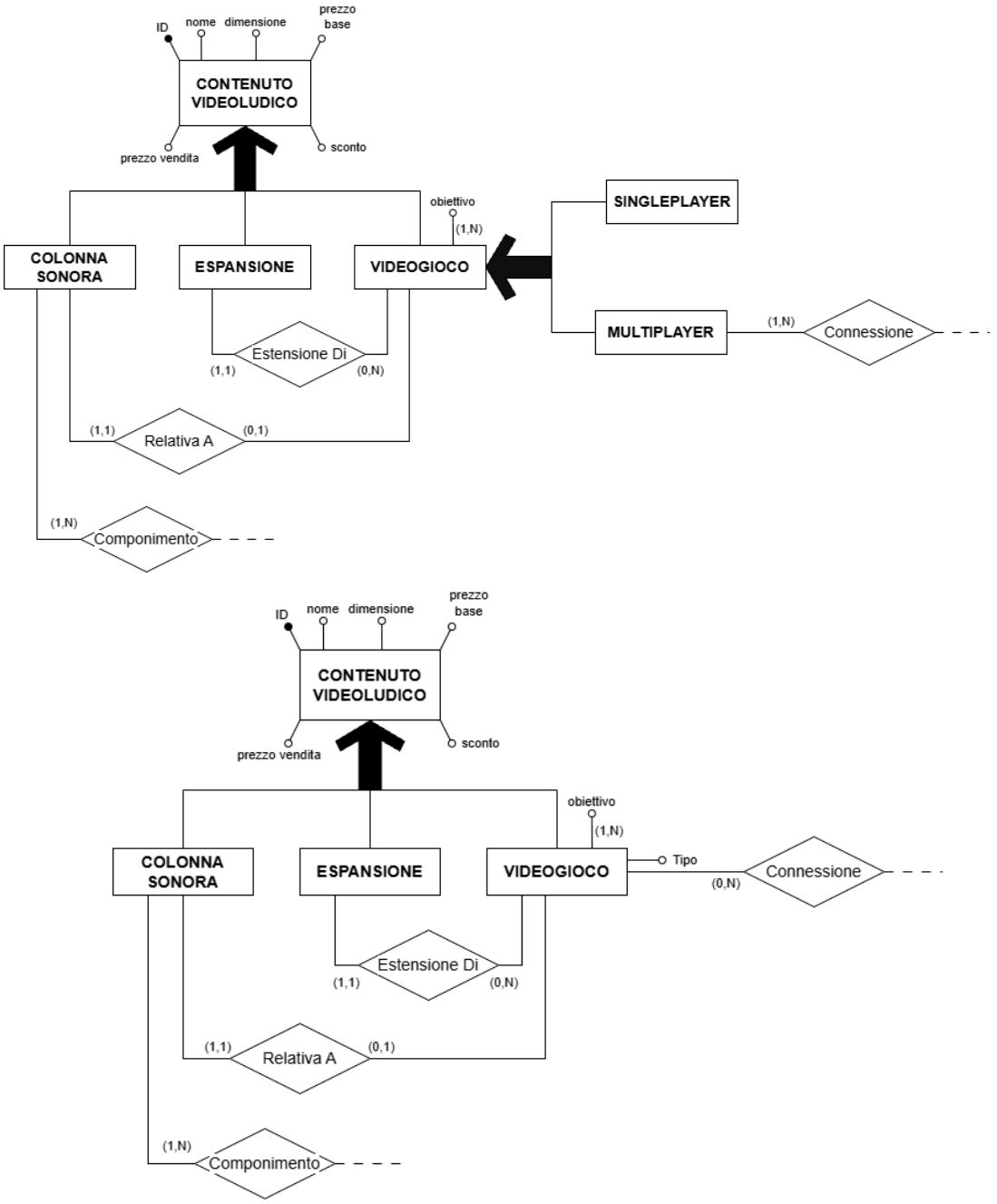


Figura 13: Analisi della generalizzazione VIDEOGIOCO

Per quanto riguarda la generalizzazione di livello superiore, possiamo applicare una soluzione ibrida. Possiamo quindi accorpare le entità COLONNA SONORA ed ESPANSIONE nel genitore: questo avviene perché poche operazioni fanno riferimento solamente alle entità figlie. Diversamente per VIDEOGIOCO, le operazioni caratterizzano

separamentamente VIDEOGIOCO e l'entità genitore, quindi introduciamo una relazione tra padre e figlio, chiamata Dettagli.
Lo schema risultante è il seguente:

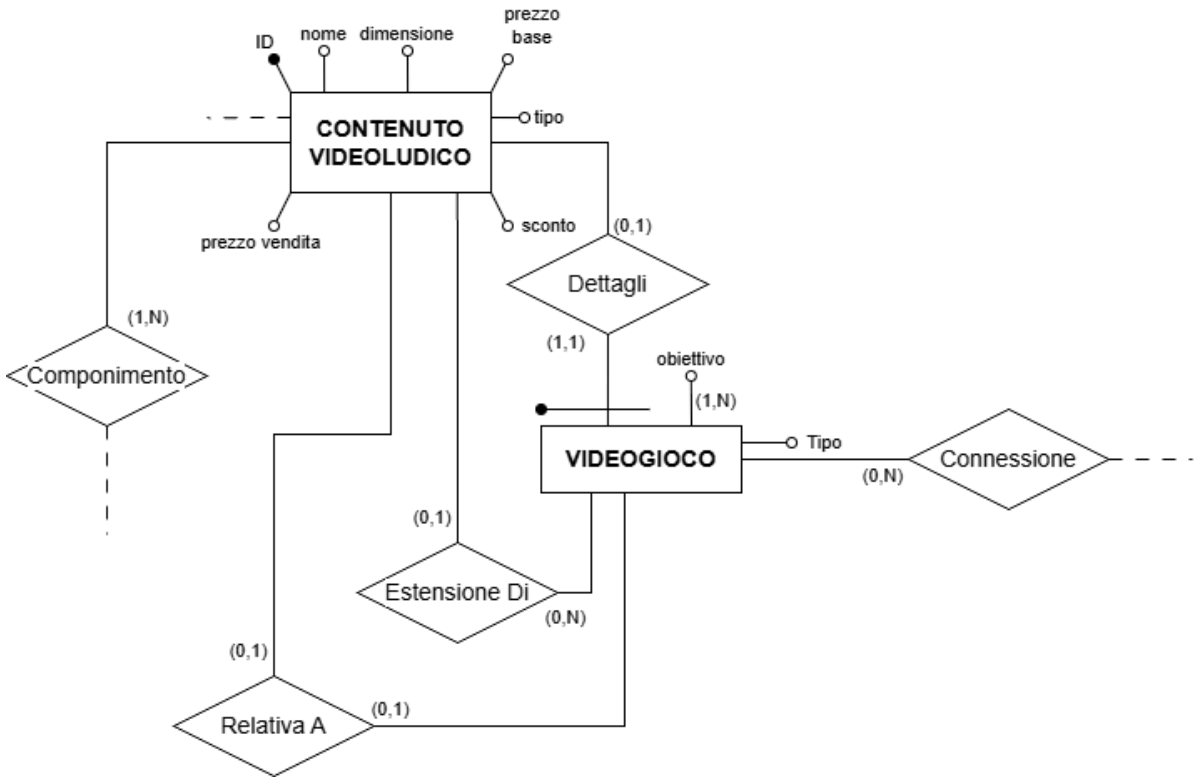


Figura 14: Analisi della generalizzazione CONTENUTO VIDEOLUDICO

Andiamo infine a studiare la generalizzazione presente in SERVER, difatti questa collasserà verso l'alto dato che le operazioni non distinguono troppo spesso le figlie dal genitore.
Quindi faremo collassare le figlie COMMUNITY e PROPRIETARIO all'interno dell'entità SERVER e le distinguiamo tramite l'attributo Tipo.
Lo schema assumerà il seguente aspetto:

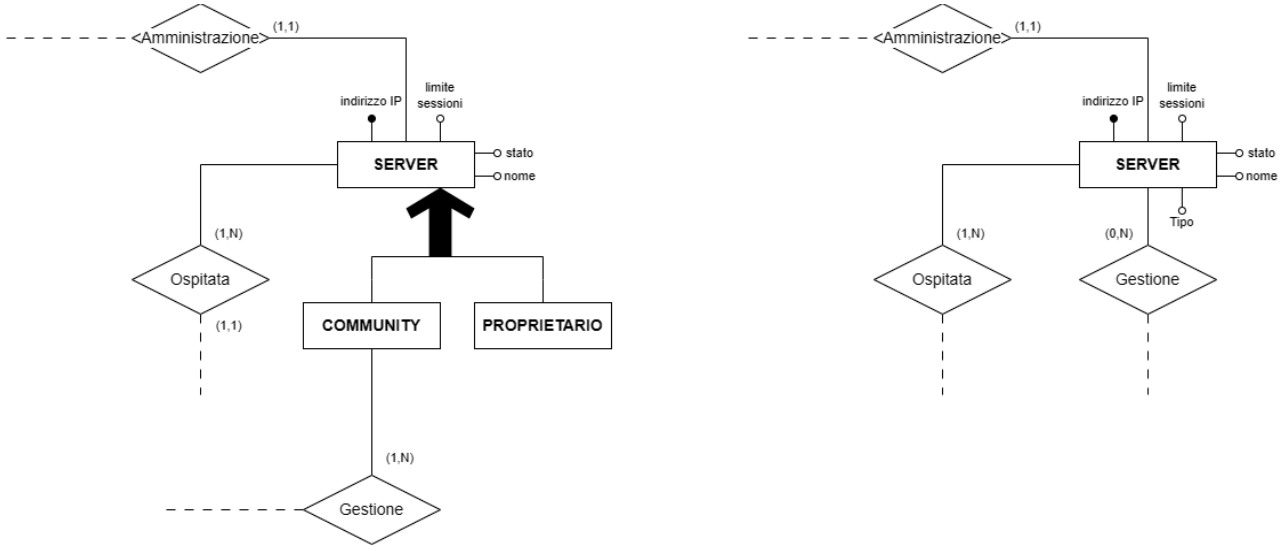


Figura 15: Analisi della generalizzazione SERVER

Accorpamento/partizionamento di relationship

L’accorpamento (operazione inversa del partizionamento) è un’operazione effettuata solitamente su associazioni con cardinalità di tipo uno a uno che porta ad unire due diverse entità. Il partizionamento orizzontale (operazione inversa dell’accorpamento) è un’operazione di decomposizione di relationship tra due entità in due (o più) relationship tra le medesime entità.

Un possibile accorpamento è relativo all’entità SESSIONE e alle relazioni Connessione e Ospitata, con conseguente sostituzione dell’entità con un’omonima relationship e rimozione delle due relazioni menzionate. Tuttavia tale azione causerebbe una perdita d’informazione non indifferente: sarebbe infatti impossibile specificare tramite lo schema che un server è in grado di ospitare più sessioni contemporaneamente.

Una possibile eliminazione degli attributi multivalore che possiamo fare è relativa all’attributo Obiettivo che ha cardinalità (1, N). Andiamo a rimuovere l’attributo relativo all'entità VIDEOGIOCO e creiamo al suo posto l’entità OBIETTIVO che si collega con VIDEOGIOCO tramite la relazione Scopo. Si identifica OBIETTIVO tramite un numero e il rispettivo videogioco.
Lo schema risultante è il seguente:

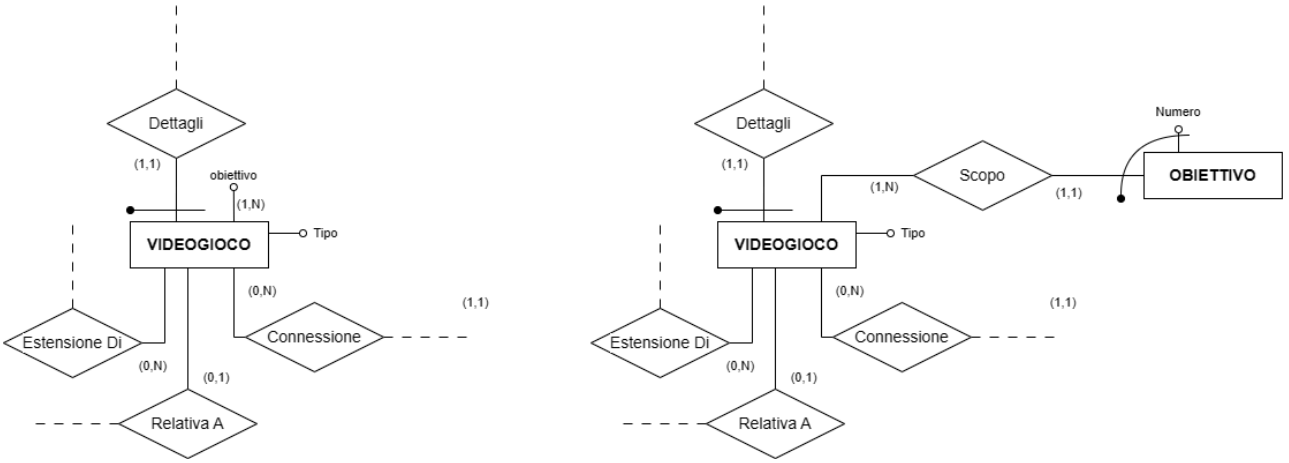


Figura 16: Eliminazione dell’attributo multivalore obiettivo

Scelta degli identificatori primari

La scelta degli identificatori principali è essenziale nelle traduzioni verso il modello relazionale poiché in tale modello le chiavi vengono utilizzate per stabilire legami tra dati in relazioni diverse.

E' necessario un intervento sull'entità SISTEMA OPERATIVO: essa è infatti identificata da 2 attributi, ed è quindi necessaria una soluzione che proponga un singolo identificatore. Per tale ragione si è deciso di creare l'attributo IdSO, identificatore di un sistema operativo. L'implementazione è la seguente:

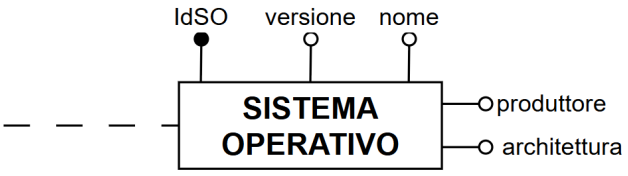


Figura 17: implementazione dell'identificatore primario in SISTEMA OPERATIVO

A seguito di tale modifica, si è giunti al termine della ristrutturazione dello schema concettuale, in quanto le altre entità presentano un solo attributo identificativo.
Fanno eccezione OBIETTIVO e BRANO, entità deboli che possiedono un identificatore esterno ed uno interno, ma che si è deciso di mantenere inalterate per il forte legame che hanno con le entità a cui sono relazionate (ossia VIDEOGIOCO e COLONNA SONORA).
Si illustra il risultato finale in Figura 18.

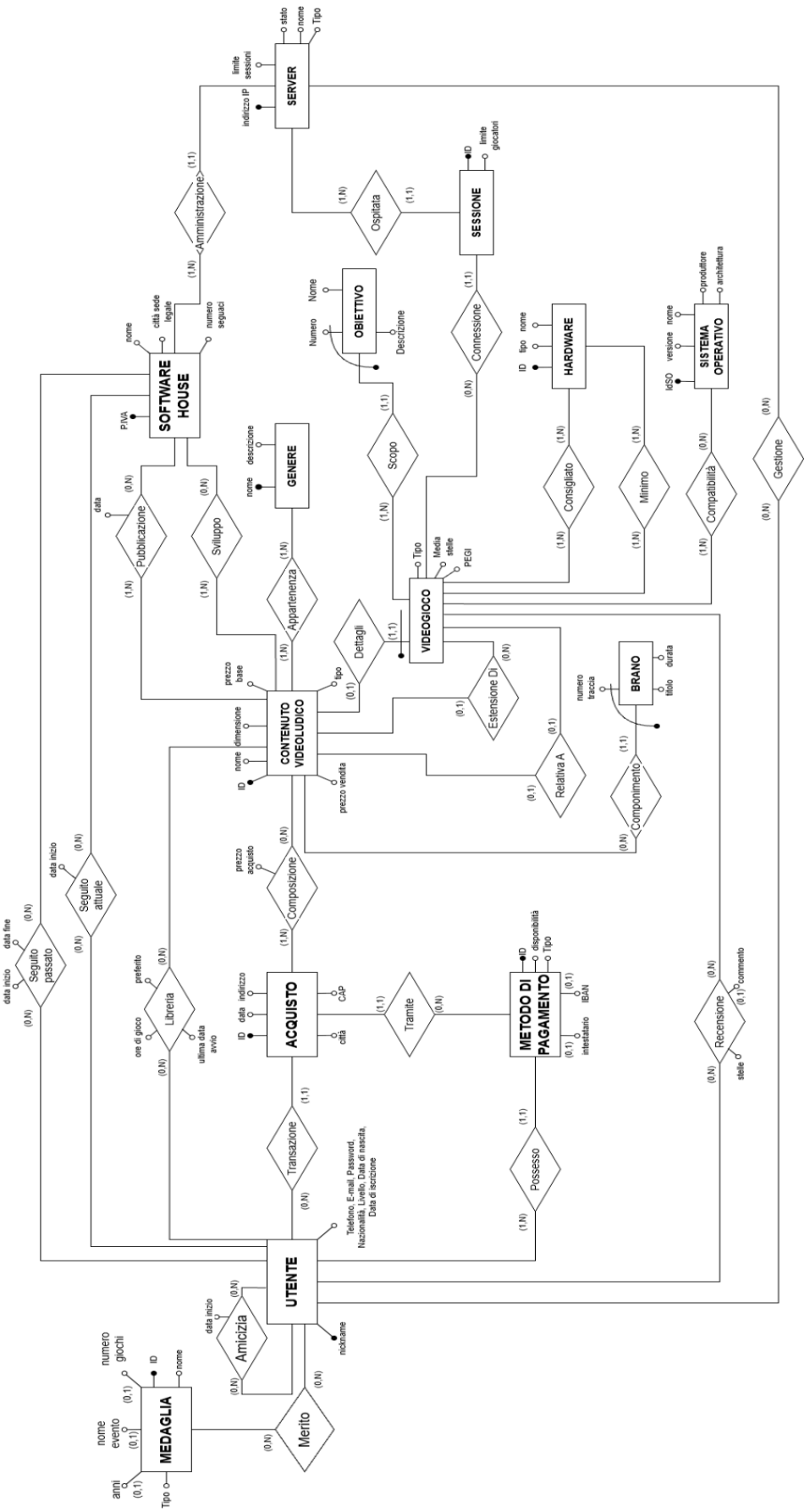


Figura 18: Schema finale ristrutturato

Modello relazionale

A partire dallo schema entità-relazione ristrutturato si costruisce uno schema logico equivalente in grado di rappresentare le medesime informazioni. Tale schema, chiamato modello relazione, costituisce il risultato della progettazione logica.

Traduzione di Entità:

- UTENTE (Nickname, Telefono, Email, Password, Nazionalità, Livello, DataDiNascita, DataDiIscrizione);

- MEDAGLIA (IdMedaglia, Nome, Tipo, Anni*, NomeEvento*, NumeroGiochi*);

- METODO DI PAGAMENTO (IdMetodo, Disponibilità, Tipo, IBAN*, Intestatario*, Proprietario)
  Vincolo d'integrità referenziale fra l'attributo Proprietario e la relazione UTENTE;

- ACQUISTO (IdAcquisto, Data, Indirizzo, CAP, Città, Utente, MetodoPagamento)
  Vincolo d'integrità referenziale tra l'attributo Utente e la relazione UTENTE e fra l'attributo MetodoPagamento e la relazione METODO DI PAGAMENTO;

- GENERE (Nome, Descrizione);

- VIDEOGIOCO (IdVideogioco, Tipo, PEGI, MediaStelle)
  Vincolo d'integrità referenziale fra l'attributo IdVideogioco e la relazione CONTENUTO VIDEOLUDICO;

- OBIETTIVO(Numero, Videogioco, Nome, Descrizione)
  Vincolo d'integrità referenziale fra l'attributo Videogioco e la relazione VIDEOGIOCO

- CONTENUTO VIDEOLUDICO (IdContenuto, Nome, Dimensione, PrezzoBase, PrezzoVendita, Tipo, EstensioneDi, RelativaA)
  Vincoli d'integrità referenziale fra l'attributo EstensioneDi e la relazione VIDEOGIOCO e fra l'attributo RelativaA e la relazione VIDEOGIOCO;

- BRANO (NumeroDiTraccia, ColonnaSonora, Titolo, Durata)
  Vincolo d'integrità referenziale tra l'attributo ColonnaSonora e la relazione CONTENUTO VIDEOLUDICO;

- HARDWARE (IdHardware, Tipo, Nome);

- SISTEMA OPERATIVO (IdSistemaOperativo, Nome, Versione, Produttore, Architettura);

- SOFTWARE HOUSE (PartitaIVA, Nome, Citta, Numero Seguaci);

- SERVER (IP, Nome, Tipo, Stato, LimiteSessioni, Amministratore)
  Vincolo d'integrità referenziale fra l'attributo Amministratore e la relazione SOFTWARE HOUSE;

- SESSIONE (IdSessione, LimiteGiocatori, Videogioco, Server)
  Vincoli d'integrità referenziale fra l'attributo Videogioco e la relazione VIDEOGIOCO e fra l'attributo Server e la relazione SERVER;

Traduzione di Relazioni:

- AMICIZIA (Utente, Amico, DataInizio)
  Vincolo di integrità referenziale tra l'attributo Utente la relazione UTENTE e tra l'attributo Amico e la relazione UTENTE;

- COMPOSIZIONE (Acquisto, Prodotto, PrezzoAcquisto)
  Vincolo d'integrità referenziale fra l'attributo Acquisto e la relazione ACQUISTO e fra l'attributo Prodotto e la relazione CONTENUTO VIDEOLUDICO;

- MERITO (Utente, Medaglia)
  Vincoli d'integrità referenziale fra l'attributo Utente e la relazione UTENTE e fra l'attributo Medaglia e la relazione MEDAGLIA;

- APPARTENENZA (Contenuto, Genere)
  Vincoli d'integrità referenziale fra l'attributo Contenuto e la relazione CONTENUTO VIDEOLUDICO e fra l'attributo Genere e la relazione GENERE;

- RECENSIONE (Utente, Videogioco, Stelle, Commento*)
  Vincoli d'integrità referenziale fra l'attributo Utente e la relazione UTENTE e fra l'attributo Videogioco e la relazione VIDEOGIOCO;

- LIBRERIA (Utente, Contenuto, OreDiGioco, UltimaDataAvvio, Preferito*)
  Vincolo d'integrità referenziale fra l'attributo Utente e la relazione UTENTE e fra l'attributo Contenuto e la relazione CONTENUTO VIDEOLUDICO

- MINIMO (Videogioco, Hardware)
  Vincoli d'integrità referenziale fra l'attributo Videogioco e la relazione VIDEOGIOCO e fra l'attributo Hardware e la relazione HARDWARE;

- CONSIGLIATO (Videogioco, Hardware)
  Vincoli d'integrità referenziale fra l'attributo Videogioco e la relazione VIDEOGIOCO e fra l'attributo Hardware e la relazione HARDWARE;

- COMPATIBILITA' (Videogioco, SistemaOperativo)
  Vincoli d'integrità referenziale fra l'attributo Videogioco e la relazione VIDEOGIOCO e fra l'attributo SistemaOperativo e la relazione SISTEMA OPERATIVO;

- SVILUPPO (Contenuto, SoftwareHouse)
  Vincoli d'integrità referenziale fra l'attributo Contenuto e la relazione CONTENUTO VIDEOLUDICO e fra l'attributo SoftwareHouse e la relazione SOFTWARE HOUSE;

- PUBBLICAZIONE (Contenuto, SoftwareHouse, DataRilascio)
  Vincoli d'integrità referenziale fra l'attributo Contenuto e la relazione CONTENUTO VIDEOLUDICO e fra l'attributo SoftwareHouse e la relazione SOFTWARE HOUSE;

- SEGUITO ATTUALE (Utente, SoftwareHouse, InizioSeguito)
  Vincoli d'integrità referenziale fra l'attributo Utente e la relazione UTENTE e fra l'attributo SoftwareHouse e la relazione SOFTWARE HOUSE;

- SEGUITO PASSATO (Utente, SoftwareHouse, InizioSeguito, FineSeguito)
  Vincoli d'integrità referenziale fra l'attributo Utente e la relazione UTENTE e fra l'attributo SoftwareHouse e la relazione SOFTWARE HOUSE;

- GESTIONE (Utente, Server)
  Vincoli d'integrità referenziale fra l'attributo Utente e la relazione UTENTE e fra l'attributo Server e la relazione SERVER;

Normalizzazione

Per valutare la qualità della base dati si procede alla sua normalizzazione.
Una relazione che non rispetta la 3FN è la relazione
ACQUISTO (IdAcquisto, Data, Indirizzo, CAP, Città, Utente, MetodoPagamento)
poiché esiste la dipendenza funzionale CAP → Città in cui CAP non è chiave.

Possiamo applicare l'algoritmo per il calcolo della 3FN:

Dipendenze funzionali individuate:

F = { IDAcquisto → Data Città CAP Indirizzo, CAP → Città }

Calcoliamo la copertura ridotta

F' = { IDAcquisto → Data, IDAcquisto → Città, IDAcquisto → CAP, IDAcquisto → Indirizzo, CAP → Città }

Rimuoviamo le ridondanze, in questo caso l'unica ridondanza è IDAcquisto → Città

F' senza ridondanza = { IDAcquisto → Data, IDAcquisto → CAP, IDAcquisto → Indirizzo, CAP → Città }

Partizioniamo F' in sottoinsiemi

F'_1 = { IDAcquisto → Data, IDAcquisto → CAP, IDAcquisto → Indirizzo }

F'_2 = { CAP → Città }

Definiamo le nuove relazioni

R_1 (IDAcquisto, Data, CAP, Indirizzo) chiamata ACQUISTO

R_2 (CAP, Città) chiamata LOCALITA’

In questo modo la relazione è in 3FN.
Volendo tradurre l’analisi appena fatta nello schema E-R:

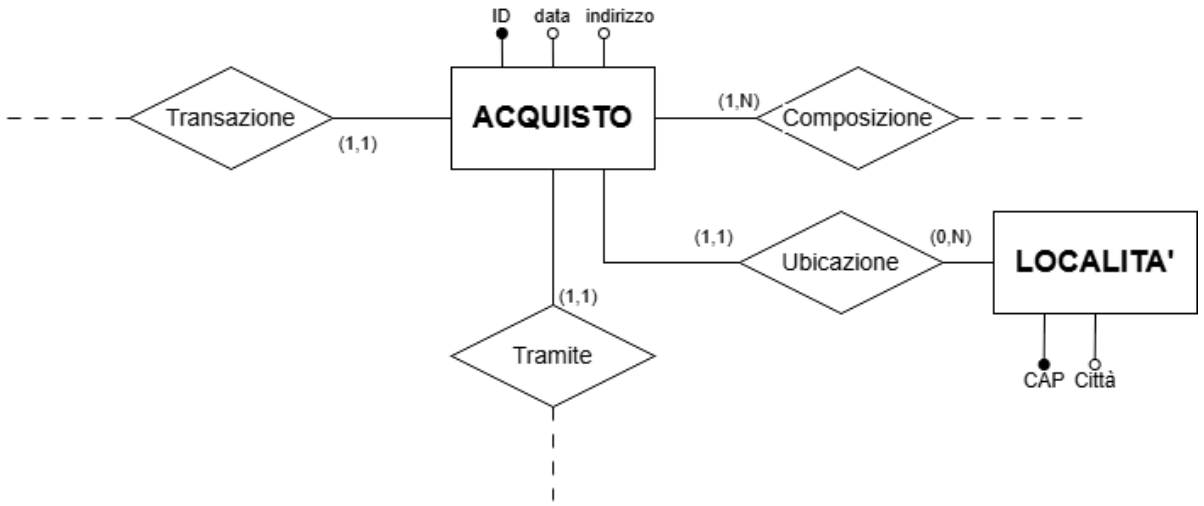


Figura 18: Schema dopo la normalizzazione

E’ stata aggiunta l’entità LOCALITA’ legata ad ACQUISTO tramite la relazione Ubicazione

Viene modificato anche il modello relazionale:

- ACQUISTO (IdAcquisto, Data, Indirizzo, CAP, Utente, MetodoPagamento)
  Vincolo d’integrità referenziale tra l’attributo Utente e la relazione UTENTE, fra l’attributo MetodoPagamento e la relazione METODO DI PAGAMENTO e fra l’attributo CAP e la relazione LOCALITA’;

- LOCALITA’ (CAP, Città)

Si vuole notare che le relazioni sono anche in BCNF, in quanto per ogni dipendenza funzionale non banale, l’implicante contiene una chiave della relazione.

Un’altra possibile anomalia è quella che riguarda l’attributo Media Stelle in VIDEOGIOCO. L’attributo non è una proprietà intrinseca del videogioco stesso ma è strettamente legato alle recensioni di quest’ultimo.
Quindi Media Stelle non dipende dalla chiave primaria IdVideogioco, per questo la relazione non è in 3FN.
Tuttavia, in questo caso, vogliamo lasciarla tale in quanto eliminare l’attributo renderebbe le operazioni **OP28** e **OP29** molto pesanti.

Implementazione delle operazioni

Creazione delle tabelle

```
CREATE TABLE Utente
(
      Nickname varchar(20) PRIMARY KEY,
      Telefono varchar (15) NOT NULL,
      Email varchar(50) NOT NULL,
      Password varchar(50) NOT NULL,
      Nazionalita varchar(30) NOT NULL,
      Livello integer default 1,
      DataDiNascita date NOT NULL,
      DataDiIscrizione date NOT NULL,
      UNIQUE(Email)
);

CREATE TABLE Medaglia
(
      IdMedaglia integer PRIMARY KEY,
      Nome varchar(50) NOT NULL,
      Tipo varchar(30) NOT NULL,
      Anni integer,
      NomeEvento varchar(50),
      NumeroGiochi integer,
      UNIQUE(Nome, Tipo)
);

CREATE TABLE MetodoDiPagamento
(
      IdMetodo integer PRIMARY KEY,
      Disponibilita decimal(10,2) default 0,
      Tipo varchar(20) NOT NULL,
      IBAN varchar(34),
```

```
      Intestatario varchar(50),
      Proprietario varchar(20) NOT NULL,
      FOREIGN KEY (Proprietario)
      REFERENCES Utente(Nickname)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

CREATE TABLE Localita

```
(
      CAP varchar(12) PRIMARY KEY,
      Citta varchar(50) NOT NULL
);
```

CREATE TABLE Acquisto

```
(
      IdAcquisto integer PRIMARY KEY,
      Data date NOT NULL,
      Indirizzo varchar(50) NOT NULL,
      CAP varchar(12) NOT NULL,
      Utente varchar(20) NOT NULL,
      MetodoPagamento integer NOT NULL,
      FOREIGN KEY (CAP)
      REFERENCES Localita(CAP)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (Utente)
      REFERENCES Utente(Nickname)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (MetodoPagamento)
      REFERENCES MetodoDiPagamento(IdMetodo)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

CREATE TABLE Genere

```
(
     Nome varchar(30) PRIMARY KEY,
     Descrizione varchar(50)
);

CREATE TABLE Videogioco
(
     IdVideogioco integer PRIMARY KEY,
     Tipo varchar(20) NOT NULL,
     PEGI integer NOT NULL,
     MediaStelle decimal(2,1) default 0.0
);

CREATE TABLE Obiettivo
(
     Numero integer,
     Videogioco integer,
     Nome varchar(20) NOT NULL,
     Descrizione varchar(200) NOT NULL,
     PRIMARY KEY(Numero, Videogioco),
     FOREIGN KEY(Videogioco)
     REFERENCES Videogioco(IdVideogioco)
          ON DELETE CASCADE
          ON UPDATE CASCADE
);

CREATE TABLE ContenutoVideoludico
(
     IdContenuto integer PRIMARY KEY,
     Nome varchar(50) NOT NULL,
     Dimensione decimal(6,2),
     PrezzoBase decimal(10,2) default 0,
     PrezzoVendita decimal(10,2) default 0,
     Tipo varchar(20) NOT NULL,
     EstensioneDi integer,
     RelativaA integer,
     FOREIGN KEY (EstensioneDi)
```

```
      REFERENCES Videogioco(IdVideogioco)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (RelativaA)
      REFERENCES Videogioco(IdVideogioco)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);

ALTER TABLE Videogioco
      ADD CONSTRAINT fk_id_videogioco
      FOREIGN KEY (IdVideogioco)
      REFERENCES ContenutoVideoludico(IdContenuto)
            ON DELETE CASCADE
            ON UPDATE CASCADE;

CREATE TABLE Brano
(
      NumeroDiTraccia integer NOT NULL,
      ColonnaSonora integer NOT NULL,
      Titolo varchar(30) NOT NULL,
      Durata time NOT NULL,
      PRIMARY KEY(NumeroDiTraccia, ColonnaSonora),
      FOREIGN KEY(ColonnaSonora)
      REFERENCES ContenutoVideoludico(IdContenuto)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);

CREATE TABLE Hardware
(
      IdHardware integer PRIMARY KEY,
      Tipo varchar(20) NOT NULL,
      Nome varchar(20) NOT NULL
);
```

```
CREATE TABLE SistemaOperativo
(
     IdSistemaOperativo integer PRIMARY KEY,
     Nome varchar(20) NOT NULL,
     Versione varchar(20) NOT NULL,
     Produttore varchar(20) NOT NULL,
     Architettura varchar(10) NOT NULL
);

CREATE TABLE SoftwareHouse
(
     PartitaIVA varchar(20) PRIMARY KEY,
     Nome varchar(50) NOT NULL,
     Citta varchar(50) NOT NULL,
     NumeroSeguaci integer default 0
);

CREATE TABLE Server
(
     IP varchar(15) PRIMARY KEY,
     Nome varchar(30) NOT NULL,
     Tipo varchar(15) NOT NULL,
     Stato varchar(20) NOT NULL,
     LimiteSessioni integer default 1,
     Amministratore varchar(12) NOT NULL,
     FOREIGN KEY (Amministratore)
     REFERENCES SoftwareHouse(PartitaIVA)
          ON DELETE CASCADE
          ON UPDATE CASCADE
);

CREATE TABLE Sessione
(
     IdSessione integer PRIMARY KEY,
     LimiteGiocatori integer NOT NULL,
     Videogioco integer NOT NULL,
     Server varchar(15) NOT NULL,
     FOREIGN KEY(Videogioco)
```

```
      REFERENCES Videogioco(IdVideogioco)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY(Server)
      REFERENCES Server(IP)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

```
CREATE TABLE Amicizia
(
      Utente varchar(20),
      Amico varchar(20),
      DataInizio date NOT NULL,
      PRIMARY KEY (Utente, Amico),
      FOREIGN KEY (Utente)
      REFERENCES Utente(Nickname)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (Amico)
      REFERENCES Utente(Nickname)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

```
CREATE TABLE Composizione
(
      Acquisto integer,
      Prodotto integer,
      PrezzoAcquisto decimal default 0.0,
      PRIMARY KEY (Acquisto, Prodotto),
      FOREIGN KEY (Acquisto)
      REFERENCES Acquisto(IdAcquisto)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (Prodotto)
      REFERENCES ContenutoVideoludico(IdContenuto)
```

```
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

```
CREATE TABLE Merito
(
      Utente varchar (20),
      Medaglia integer,
      PRIMARY KEY (Utente, Medaglia),
      FOREIGN KEY (Utente)
      REFERENCES Utente(Nickname)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (Medaglia)
      REFERENCES Medaglia(IdMedaglia)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

```
CREATE TABLE Appartenenza
(
      Contenuto integer,
      Genere varchar(30),
      PRIMARY KEY (Contenuto, Genere),
      FOREIGN KEY (Contenuto)
      REFERENCES ContenutoVideoludico(IdContenuto)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (Genere)
      REFERENCES Genere(Nome)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

```
CREATE TABLE Recensione
(
```

```
      Utente varchar(20),
      Videogioco integer,
      Stelle decimal(2,1) NOT NULL,
      Commento varchar(200),
      PRIMARY KEY (Utente, Videogioco),
      FOREIGN KEY (Utente)
      REFERENCES Utente(Nickname)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (Videogioco)
      REFERENCES Videogioco(IdVideogioco)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

CREATE TABLE Libreria

```
(
      Utente varchar(20),
      Contenuto integer,
      OreDiGioco time default '00:00:00',
      UltimaDataAvvio date default NULL,
      Preferito boolean default FALSE,
      PRIMARY KEY (Utente, Contenuto),
      FOREIGN KEY (Utente)
      REFERENCES Utente(Nickname)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (Contenuto)
      REFERENCES ContenutoVideoludico(IdContenuto)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

CREATE TABLE Minimo

```
(
      Videogioco integer,
      Hardware integer,
      PRIMARY KEY (Videogioco, Hardware),
```

```
      FOREIGN KEY (Videogioco)
      REFERENCES Videogioco(IdVideogioco)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (Hardware)
      REFERENCES Hardware(IdHardware)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

CREATE TABLE Consigliato

```
(
      Videogioco integer,
      Hardware integer,
      PRIMARY KEY (Videogioco, Hardware),
      FOREIGN KEY (Videogioco)
      REFERENCES Videogioco(IdVideogioco)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (Hardware)
      REFERENCES Hardware(IdHardware)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

CREATE TABLE Compatibilita

```
(
      Videogioco integer,
      SistemaOperativo integer,
      PRIMARY KEY (Videogioco, SistemaOperativo),
      FOREIGN KEY (Videogioco)
      REFERENCES Videogioco(IdVideogioco)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (SistemaOperativo)
      REFERENCES SistemaOperativo(IdSistemaOperativo)
            ON DELETE CASCADE
```

```
            ON UPDATE CASCADE
);

CREATE TABLE Sviluppo
(
      Contenuto integer,
      SoftwareHouse varchar(20),
      PRIMARY KEY (Contenuto, SoftwareHouse),
      FOREIGN KEY (Contenuto)
      REFERENCES ContenutoVideoludico(IdContenuto)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (SoftwareHouse)
      REFERENCES SoftwareHouse(PartitaIVA)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);

CREATE TABLE Pubblicazione
(
      Contenuto integer,
      SoftwareHouse varchar(20),
      DataRilascio date NOT NULL,
      PRIMARY KEY (Contenuto, SoftwareHouse),
      FOREIGN KEY (Contenuto)
      REFERENCES ContenutoVideoludico(IdContenuto)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (SoftwareHouse)
      REFERENCES SoftwareHouse(PartitaIVA)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);

CREATE TABLE SeguitoAttuale
(
      Utente varchar(20),
```

```
      SoftwareHouse varchar(20),
      InizioSeguito date NOT NULL,
      PRIMARY KEY (Utente, SoftwareHouse),
      FOREIGN KEY (Utente)
      REFERENCES Utente(Nickname)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (SoftwareHouse)
      REFERENCES SoftwareHouse(PartitaIVA)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

CREATE TABLE SeguitoPassato

```
(
      Utente varchar(20),
      SoftwareHouse varchar(20),
      InizioSeguito date NOT NULL,
      FineSeguito date NOT NULL,
      PRIMARY KEY (Utente, SoftwareHouse),
      FOREIGN KEY (Utente)
      REFERENCES Utente(Nickname)
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (SoftwareHouse)
      REFERENCES SoftwareHouse(PartitaIVA)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

CREATE TABLE Gestione

```
(
      Utente varchar(20),
      Server varchar(15),
      PRIMARY KEY (Utente, Server),
      FOREIGN KEY (Utente)
      REFERENCES Utente(Nickname)
```

```
            ON DELETE CASCADE
            ON UPDATE CASCADE,
      FOREIGN KEY (Server)
      REFERENCES Server(IP)
            ON DELETE CASCADE
            ON UPDATE CASCADE
);
```

Implementazione dei vincoli

- Ogni utente deve avere un'e-mail che rispetti una certa sintassi e una data di nascita che non superi il giorno attuale:

```
ALTER TABLE Utente
ADD CONSTRAINT chk_dati_utente
CHECK (
      Email LIKE '%@%.%' AND
      DataDiNascita < CURRENT_DATE
);
```

- Le medaglie possono essere solo di 3 tipi, ogni tipo ha il suo dominio che va definito:

```
ALTER TABLE Medaglia
ADD CONSTRAINT chk_medaglie_acquisibili
CHECK (
      (Tipo = 'Anzianità') OR
      (Tipo = 'Numero giochi' AND (NumeroGiochi IN (10, 25, 50) OR
      NumeroGiochi%100 = 0)) OR
      (Tipo = 'Evento')
);
```

- Il metodo di pagamento può essere o un conto bancario o il saldo interno e in entrambi i casi deve avere una disponibilità superiore o pari a 0.
Infine il nome dell'intestatario deve chiaramente essere costituito da sole lettere:

```
ALTER TABLE MetodoDiPagamento
ADD CONSTRAINT chk_validazione_pagamento
CHECK(
      (Tipo = 'Conto bancario' OR Tipo = 'Saldo') AND
      Disponibilita >= 0 AND
      (Intestatario ~ '^[[:alpha:] ]+$')
);
```

- Il Videogioco ha il campo PEGI che può assumere solo alcuni valori specifici che fanno riferimento alle età minime (3, 7, 12, 16 e 18 anni):

```
ALTER TABLE Videogioco
ADD CONSTRAINT chk_limiti_eta_e_tipo
CHECK(
      PEGI IN (3, 7, 12, 16, 18) AND
      (Tipo = 'Singleplayer' OR Tipo = 'Multiplayer')
);
```

- La valutazione media di un Videogioco può andare da 1 a 5 stelle. Un videogioco ha valutazione media pari a 0 se ancora non ha ricevuto alcuna valutazione:

```
ALTER TABLE Videogioco
ADD CONSTRAINT chk_range_valutazione_media
CHECK(
      (MediaStelle <= 5 AND MediaStelle >= 1) OR MediaStelle = 0
);
```

- Un Contenuto videoludico può avere solo un prezzo positivo e deve essere categorizzato da uno dei 3 tipi: videogioco, colonna sonora o espansione:

```
ALTER TABLE ContenutoVideoludico
ADD CONSTRAINT chk_classificazione_contenuti_videoludici
CHECK(
      PrezzoBase >= 0 AND PrezzoVendita >= 0 AND
      Tipo IN ('Videogioco', 'Colonna sonora', 'Espansione')
);
```

- L'Hardware può avere solo 3 valori possibili per il tipo: CPU, RAM, GPU:

```
ALTER TABLE Hardware
ADD CONSTRAINT chk_componenti_fisiche
CHECK(
      Tipo IN ('CPU', 'RAM', 'GPU')
);
```

- Il numero dei seguaci di una Software house non può essere negativo:

```
ALTER TABLE SoftwareHouse
ADD CONSTRAINT chk_dati_software_house
CHECK(
      NumeroSeguaci >= 0
);
```

- L’indirizzo IP (IPv4) è composto al limite da 15 caratteri (xxx.xxx.xxx.xxx) che sono tutti separati da punti, inoltre il Server può distinguersi in Community e Proprietario:

```
ALTER TABLE Server
ADD CONSTRAINT chk_ip_server
CHECK(
      IP LIKE '%.%.%.%' AND
      Tipo IN ('Community', 'Proprietario')
);
```

- Il prezzo di acquisto di una serie di contenuti videoludici che compongono l’ordine non può avere valori negativi:

```
ALTER TABLE Composizione
ADD CONSTRAINT chk_prezzo_valido
CHECK(
      PrezzoAcquisto >= 0
);
```

- Una Recensione di un videogioco si effettua con una valutazione in stelle, che deve essere in un range che va da 1 a 5, a multipli di 0.5:

```
ALTER TABLE Recensione
ADD CONSTRAINT chk_range_valutazione
CHECK(
      Stelle IN (1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5)
);
```

Creazione delle query

OP1: Registrazione di un utente

```
CREATE OR REPLACE FUNCTION Registra_Utente(
u_nick varchar,
u_tel varchar,
u_email varchar,
u_psw varchar,
u_naz varchar,
u_nasc date)
```

```
RETURNS void AS $$
BEGIN
IF NOT EXISTS (SELECT 1 FROM Utente WHERE Nickname=u_nick) AND NOT
EXISTS (SELECT 1 FROM Utente WHERE Email=u_email) THEN
      INSERT INTO Utente(Nickname, Telefono, Email, Password,
      Nazionalita, DataDiNascita, DataDiIscrizione)
      VALUES(u_nick, u_tel, u_email, u_psw, u_naz, u_nasc,
      CURRENT_DATE);

      INSERT INTO MetodoDiPagamento(IdMetodo, Tipo, Proprietario)
      VALUES((
            SELECT COALESCE(MAX(IdMetodo), 0) + 1
            FROM MetodoDiPagamento), 'Saldo', u_nick);

      RAISE NOTICE 'Utente % registrato con successo', u_nick;
ELSE
      RAISE NOTICE 'Errore: Utente e/o Email già registrati.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP2: Eliminazione di un utente

```
CREATE OR REPLACE FUNCTION Elimina_Utente (
      i_nick varchar)
RETURNS void AS $$
DECLARE
      utente_individuato boolean;
BEGIN
SELECT EXISTS (
      SELECT 1
      FROM Utente
      WHERE Nickname=i_nick
) INTO utente_individuato;
IF utente_individuato THEN
      DELETE FROM Utente
```

```
      WHERE Nickname=i_nick;
      RAISE NOTICE 'Utente % rimosso con successo', i_nick;
ELSE
      RAISE NOTICE 'Errore: utente non individuato!';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP3: Accesso di un utente

```
CREATE OR REPLACE FUNCTION Accesso_Utente(
      i_nick varchar,
      i_pass varchar)
RETURNS boolean AS $$
DECLARE
      utente_valido boolean;
BEGIN
SELECT EXISTS(
      SELECT 1
      FROM Utente
      WHERE Nickname = i_nick AND Password=i_pass
) INTO utente_valido;

IF utente_valido THEN
      RAISE NOTICE 'Utente % è acceduto con successo', i_nick;
ELSE
      RAISE NOTICE 'Credenziali errate!';
END IF;
RETURN utente_valido;
END;
$$ LANGUAGE plpgsql;
```

OP4: Aggiunta di un amico

```
CREATE OR REPLACE FUNCTION Aggiunta_Amico(
      u_nick varchar,
      a_nick varchar)
```

```
RETURNS void AS $$
DECLARE
      amicizia boolean :=FALSE;
      utente_valido boolean;
      amico_valido boolean;
BEGIN
SELECT EXISTS(
      SELECT 1
      FROM Utente
      WHERE Nickname = u_nick) INTO utente_valido;
SELECT EXISTS(
      SELECT 1
      FROM Utente
      WHERE Nickname = a_nick) INTO amico_valido;
SELECT EXISTS(
      SELECT 1
      FROM Amicizia
      WHERE (u_nick = Utente AND a_nick = Amico) OR (u_nick = Amico
      AND a_nick = Utente)) INTO amicizia;

--Condizione per la quale un utente non può essere amico di se
stesso
IF (u_nick=a_nick) THEN
      RAISE NOTICE 'Errore: l'utente sta cercando di aggiungere se
      stesso.';
      RETURN;
END IF;
IF amicizia THEN
      RAISE NOTICE 'Utente % e % sono già amici.', u_nick, a_nick;
ELSE
      IF (utente_valido AND amico_valido) THEN
            INSERT INTO Amicizia(Utente, Amico, DataInizio)
            VALUES (u_nick, a_nick, CURRENT_DATE);
            INSERT INTO Amicizia(Utente, Amico, DataInizio)
            VALUES (a_nick,u_nick, CURRENT_DATE);
            RAISE NOTICE 'Utente % e % sono amici', u_nick, a_nick;
      ELSE
```

```
            RAISE NOTICE 'La richiesta di amicizia è stata
            rifiutata (uno degli utenti non è valido).';
      END IF;
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP5: Rimozione di un amico

```
CREATE OR REPLACE FUNCTION Rimuovi_Amico(
      u_nick varchar,
      a_nick varchar)
RETURNS void AS $$
DECLARE
      amicizia boolean;
      utente_valido boolean;
      amico_valido boolean;
BEGIN
SELECT EXISTS(
      SELECT 1
      FROM Utente
      WHERE Nickname = u_nick) INTO utente_valido;
SELECT EXISTS(
      SELECT 1
      FROM Utente
      WHERE Nickname = a_nick) INTO amico_valido;
SELECT EXISTS(
      SELECT 1
      FROM Amicizia
      WHERE (u_nick = Utente AND a_nick = Amico) OR (u_nick = Amico
AND a_nick = Utente)) INTO amicizia;

IF amicizia = FALSE THEN
      RAISE NOTICE 'Utente % e % non sono amici, impossibile
      eseguire la rimozione.', u_nick, a_nick;
ELSE
      IF (utente_valido AND amico_valido) THEN
```

```
            DELETE FROM Amicizia
            WHERE (Utente = u_nick AND Amico = a_nick) OR (Utente =
            a_nick AND Amico = u_nick);
            RAISE NOTICE 'Utente % e % non sono più amici.', u_nick,
            a_nick;
      ELSE
            RAISE NOTICE 'Uno dei due utenti non esiste.';
      END IF;
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP6: Utente aumenta di livello

```
CREATE OR REPLACE FUNCTION Aumento_Livello (u_nick varchar)
RETURNS void AS $$
DECLARE
      livello_attuale integer;
BEGIN
SELECT Livello INTO livello_attuale
FROM Utente
WHERE Nickname=u_nick;
IF livello_attuale IS NOT NULL THEN
      UPDATE Utente
      SET Livello = livello_attuale + 1
      WHERE Nickname=u_nick;
      RAISE NOTICE 'Utente % aumentato dal livello % al livello
%!', u_nick, livello_attuale, livello_attuale + 1;
ELSE
      RAISE NOTICE 'Errore, utente inesistente.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP7: Utente registra il proprio conto bancario

```
CREATE OR REPLACE FUNCTION Registrazione_Conto (
      c_disponibilita decimal,
```

```
      c_IBAN varchar(34),
      c_intestatario varchar,
      c_proprietario varchar)
RETURNS void AS $$
DECLARE
      nuovo_id integer;
BEGIN
IF EXISTS (SELECT 1 FROM Utente WHERE Nickname=c_proprietario)
THEN
      SELECT COALESCE(Max(IdMetodo), 0) + 1 INTO nuovo_id
      FROM MetodoDiPagamento;
      INSERT INTO MetodoDiPagamento(IdMetodo, Disponibilita, Tipo,
      IBAN, Intestatario, Proprietario)
      VALUES(nuovo_id, c_disponibilita, 'Conto bancario', c_IBAN,
      c_intestatario, c_proprietario);
      RAISE NOTICE 'Metodo aggiunto a %', c_proprietario;
ELSE
      RAISE NOTICE 'Utente non individuato';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP8: Visualizzazione del profilo utente

```
CREATE OR REPLACE FUNCTION visualizza_dati_utente(
      u_nick varchar)
RETURNS TABLE (
      NicknameUtente varchar,
      NazionalitaUtente varchar,
      LivelloUtente integer,
      DataDiIscrizioneUtente date)
AS $$
BEGIN
RETURN QUERY
      SELECT Nickname, Nazionalita, Livello, DataDiIscrizione
      FROM Utente
      WHERE Nickname=u_nick;
END;
```

```
$$ LANGUAGE plpgsql;
```

OP9: Aggiungere all'utente una medaglia

```
CREATE OR REPLACE FUNCTION Aggiunta_Medaglia(
      u_nick varchar,
      id_medaglia integer)
RETURNS void AS $$
BEGIN
IF EXISTS (SELECT 1 FROM Utente WHERE Nickname=u_nick) AND EXISTS
(SELECT 1 FROM Medaglia WHERE IdMedaglia=id_medaglia) THEN
      IF NOT EXISTS (SELECT 1 FROM Merito WHERE Nickname=u_nick AND
      Medaglia=id_medaglia) THEN
            INSERT INTO Merito(Utente, Medaglia)
            VALUES (u_nick, id_medaglia);
      ELSE
            RAISE NOTICE 'L'utente possiede già la medaglia!';
      END IF;
ELSE
      RAISE NOTICE 'Errore: utente/medaglia non trovato/a.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP10: Pubblicazione di un contenuto

Vista la complessità dell'operazione, si è deciso di definire più funzioni per ciascuna delle sotto-operazioni da eseguire. Si è infine voluta integrare una funzione per gestire l'intero meccanismo.

- **Pubblicazione di un contenuto videoludico**

```
CREATE OR REPLACE FUNCTION Pubblicazione_Contenuto(
      c_titolo varchar,
      c_dimensione decimal,
      c_prezzobase decimal,
      c_tipo varchar,
      sh_sviluppo character(11)[],
      sh_pubblicazione character(11)[],
```

```
     c_genere varchar[])
RETURNS integer AS $$
DECLARE
     c_id integer;
     i integer;
BEGIN
-- Generazione dell'ID
SELECT COALESCE(MAX(IdContenuto), 0) + 1 INTO c_id
FROM ContenutoVideoludico;

-- Inserimento del contenuto
INSERT INTO ContenutoVideoludico(IdContenuto, Nome, Dimensione,
PrezzoBase, PrezzoVendita, Tipo)
VALUES (c_id, c_titolo, c_dimensione, c_prezzobase, c_prezzobase,
c_tipo);

-- Inserimento degli sviluppatori e degli editori
IF sh_sviluppo IS NOT NULL THEN
     FOR i IN 1..array_length(sh_sviluppo, 1) LOOP
          INSERT INTO Sviluppo(Contenuto, SoftwareHouse)
          VALUES (c_id, sh_sviluppo[i]);
     END LOOP;
END IF;

IF sh_pubblicazione IS NOT NULL THEN
     FOR i IN 1..array_length(sh_pubblicazione, 1) LOOP
          INSERT INTO Pubblicazione(Contenuto, SoftwareHouse,
          DataRilascio)
          VALUES (c_id, sh_pubblicazione[i], CURRENT_DATE);
     END LOOP;
END IF;

-- Inserimento dei generi
IF c_genere IS NOT NULL THEN
     FOR i IN 1..array_length(c_genere, 1) LOOP
          INSERT INTO Appartenenza(Contenuto, Genere)
          VALUES (c_id, c_genere[i]);
```

```
      END LOOP;
END IF;
RETURN c_id;
END;
$$ LANGUAGE plpgsql;
```

- **Pubblicazione di un videogioco**

```
CREATE OR REPLACE FUNCTION Pubblicazione_Videogioco (
      idvideogioco integer,
      v_tipo varchar,
      v_PEGI integer,
      so_id integer[],
      hw_minimo integer[],
      hw_consigliato integer[]
)
RETURNS void AS $$
DECLARE
      i integer;
BEGIN
-- Inserimento nella tabella videogioco
INSERT INTO Videogioco(IdVideogioco, Tipo, PEGI)
VALUES (idvideogioco, v_tipo, v_PEGI);

-- Sistemi operativi compatibili
IF so_id IS NOT NULL THEN
      FOR i IN 1..array_length(so_id, 1) LOOP
            INSERT INTO Compatibilita (Videogioco,
            SistemaOperativo)
            VALUES(idvideogioco, so_id[i]);
      END LOOP;
END IF;

-- Requisiti Hardware
IF hw_minimo IS NOT NULL THEN
      FOR i IN 1..array_length(hw_minimo, 1) LOOP
            INSERT INTO Minimo(Videogioco, Hardware)
            VALUES (idvideogioco, hw_minimo[i]);
```

```
      END LOOP;
END IF;

IF hw_consigliato IS NOT NULL THEN
      FOR i IN 1..array_length(hw_consigliato, 1) LOOP
            INSERT INTO Consigliato(Videogioco, Hardware)
            VALUES (idvideogioco, hw_consigliato[i]);
      END LOOP;
END IF;
END;
$$ LANGUAGE plpgsql;
```

- **Pubblicazione di un'espansione**

```
CREATE OR REPLACE FUNCTION Pubblicazione_Espansione (
      idespansione integer,
      idvideogioco integer)
RETURNS void AS $$
BEGIN
-- Verifica che il rispettivo videogioco esiste
IF EXISTS (SELECT 1 FROM Videogioco WHERE
IdVideogioco=idvideogioco) THEN
      UPDATE ContenutoVideoludico
      SET EstensioneDi=idvideogioco
      WHERE IdContenuto=idespansione;
ELSE
      RAISE EXCEPTION 'Il videogioco non esiste.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

- **Pubblicazione di una colonna sonora**

```
CREATE OR REPLACE FUNCTION Pubblicazione_ColonnaSonora (
      idcolonnasonora integer,
      idvideogioco integer,
      titoli varchar[],
      durate time[])
RETURNS void AS $$
```

```
DECLARE
     i integer;
BEGIN
-- Verifica che il videogioco esista
IF EXISTS(SELECT 1 FROM Videogioco WHERE
IdVideogioco=idvideogioco) THEN
     UPDATE ContenutoVideoludico
     SET RelativaA = idvideogioco
     WHERE IdContenuto=idcolonnasonora;

     --Inserimento di brani nella colonna sonora
     IF titoli IS NOT NULL AND durate IS NOT NULL AND
     array_length(titoli, 1)=array_length(durate, 1) THEN
          FOR i IN 1..array_length(titoli, 1) LOOP
               INSERT INTO Brano(Titolo, Durata, ColonnaSonora)
               VALUES (titoli[i], durate[i], idcolonnasonora);
          END LOOP;
     ELSE
          RAISE NOTICE 'Errore con brani e/o durate.';
     END IF;
ELSE
     RAISE EXCEPTION 'Il videogioco non esiste.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

- **Gestore delle pubblicazioni**

```
CREATE OR REPLACE FUNCTION Gestione_Pubblicazione(
     -- Parametri Base
     c_titolo varchar,
     c_dimensione decimal,
     c_prezzobase decimal,
     c_tipo varchar,
     sh_sviluppo character(11)[],
     sh_pubblicazione character(11)[],
     c_generi varchar[],
```

```
      -- Parametri Videogioco
      v_pegi integer DEFAULT NULL,
      v_tipo_gioco varchar DEFAULT NULL,
      v_so_id integer[] DEFAULT NULL,
      v_hw_minimo integer[] DEFAULT NULL,
      v_hw_consigliato integer[] DEFAULT NULL,
      -- Parametri Espansione/Colonne Sonore
      ecs_id_videogioco integer DEFAULT NULL,
      ecs_brani_titoli varchar[] DEFAULT NULL,
      ecs_brani_durate time[] DEFAULT NULL
)
RETURNS void AS $$
DECLARE
      id_generato integer;
BEGIN
--  Creazione del contenuto base
id_generato := Pubblicazione_Contenuto(c_titolo, c_dimensione,
c_prezzobase, c_tipo, sh_sviluppo, sh_pubblicazione, c_generi);

-- Gestione del tipo
CASE c_tipo
      -- Videogioco
      WHEN 'Videogioco' THEN
      PERFORM Pubblicazione_Videogioco(id_generato, v_tipo_gioco,
      v_pegi, v_so_id, v_hw_minimo, v_hw_consigliato);
      -- Espansione
      WHEN 'Espansione' THEN
      PERFORM Pubblicazione_Espansione(id_generato,
      ecs_id_videogioco);
      -- Colonna sonora
      WHEN 'Colonna sonora' THEN
      PERFORM Pubblicazione_ColonnaSonora(id_generato,
      ecs_id_videogioco, ecs_brani_titoli, ecs_brani_durate);
END CASE;
END;
$$ LANGUAGE plpgsql;
```

OP11: Rimozione di un contenuto videoludico

```
CREATE OR REPLACE FUNCTION Rimozione_Contenuto_Videoludico(
     id integer)
RETURNS void AS $$
DECLARE
     rim_valida boolean;
BEGIN
SELECT EXISTS(SELECT 1 FROM ContenutoVideoludico WHERE id =
IdContenuto) INTO rim_valida;

IF rim_valida THEN
     DELETE FROM ContenutoVideoludico
     WHERE IdContenuto = id;
     RAISE NOTICE 'Il contenuto è stato eliminato';
ELSE
     RAISE NOTICE 'Errore: il contenuto non è stato trovato';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP12: Acquisto di contenuti

```
CREATE OR REPLACE FUNCTION Acquisto_Contenuti (
     u_nick varchar,
     a_indirizzo varchar,
     a_CAP varchar(12),
     c_id integer[],
     mp_id integer
)
RETURNS void AS $$
DECLARE
     prezzo_singolo decimal;
     prezzo_totale decimal :=0;
     id_acquisto integer;
     saldo_attuale decimal;
     i integer;
```

```
BEGIN
--Verifica che l'utente esiste, che il metodo di pagamento è valido
ed è dell'utente e che il CAP è corretto
IF EXISTS (SELECT 1 FROM Utente WHERE Nickname=u_nick) AND EXISTS
(SELECT 1 FROM MetodoDiPagamento WHERE IdMetodo=mp_id AND
Proprietario=u_nick) AND EXISTS (SELECT 1 FROM Localita WHERE
CAP=a_CAP) THEN
-- Calcolo del totale dell'ordine e verifica della validità degli ID
dei contenuti
     FOR i IN 1..array_length(c_id, 1) LOOP
          SELECT PrezzoVendita INTO prezzo_singolo
          FROM ContenutoVideoludico
          WHERE IdContenuto = c_id[i];

          IF prezzo_singolo IS NULL THEN
               RAISE EXCEPTION 'Il prodotto con ID % non
               esiste.', c_id[i];
          END IF;
     prezzo_totale := prezzo_totale + prezzo_singolo;
     END LOOP;
-- Verifica disponibilità economica dell'utente
     SELECT Disponibilita INTO saldo_attuale
     FROM MetodoDiPagamento
     WHERE IdMetodo = mp_id AND Proprietario = u_nick;

     IF saldo_attuale<prezzo_totale THEN
          RAISE EXCEPTION 'Saldo insufficiente';
     END IF;
-- Inserimento di una sola tupla in acquisto
     SELECT COALESCE(MAX(IdAcquisto), 0) + 1 INTO id_acquisto FROM
     Acquisto;
     INSERT INTO Acquisto(IdAcquisto, Data, Indirizzo, CAP,
     Utente, MetodoPagamento)
     VALUES(id_acquisto, CURRENT_DATE, a_indirizzo, a_cap, u_nick,
     mp_id);
-- Scalo dal metodo di pagamento
     UPDATE MetodoDiPagamento
```

```
      SET Disponibilita = Disponibilita-prezzo_totale
      WHERE IdMetodo=mp_id;
-- Inserimento di N tuple in Composizione e Libreria
      FOR i IN 1..array_length(c_id, 1) LOOP
      -- Controllo che la tupla non esista già in Libreria
            IF NOT EXISTS (SELECT 1 FROM Libreria WHERE
            Utente=u_nick AND Contenuto=c_id[i]) THEN
            -- Recupero del prezzo di vendita del prodotto
                  SELECT PrezzoVendita INTO prezzo_singolo
                  FROM ContenutoVideoludico
                  WHERE IdContenuto=c_id[i];
            -- Aggiunta in Composizione
                  INSERT INTO Composizione(Acquisto, Prodotto,
                  PrezzoAcquisto)
                  VALUES(id_acquisto, c_id[i], prezzo_singolo);
            -- Aggiunta in Libreria
                  INSERT INTO Libreria(Utente, Contenuto)
                  VALUES(u_nick, c_id[i]);
            END IF;
      END LOOP;
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP13: Visualizzazione del videogioco più acquistato nel negozio

Per la seguente interrogazione è necessario prima creare una vista, in quanto non sarebbe possibile utilizzare una funzione nidificata nella clausola HAVING. La vista salva il numero di acquisti di tutti i videogiochi, invece la query trova il massimo tra il numero di acquisti di tutti i videogiochi.

```
CREATE VIEW ClassificaVendite (IdContenuto, Nome, Tipo,
TotaleVendite) AS
SELECT C.IdContenuto, C.Nome, C.Tipo,
COUNT(Comp.Prodotto)::integer AS TotaleVendite
FROM ContenutoVideoludico C, Composizione Comp
WHERE C.IdContenuto=Comp.Prodotto
GROUP BY C.IdContenuto, C.Nome, C.Tipo
```

```
CREATE OR REPLACE FUNCTION Gioco_Piu_Acquistato()
RETURNS TABLE(IdVideogioco integer, Titolo varchar, Vendite
integer)AS $$
BEGIN
RETURN QUERY
SELECT IdContenuto, Nome, TotaleVendite
FROM ClassificaVendite
WHERE Tipo = 'Videogioco' AND TotaleVendite >= ALL (
      SELECT TotaleVendite
      FROM ClassificaVendite
      WHERE Tipo='Videogioco'
);
END;
$$ LANGUAGE plpgsql;
```

OP14: Utente inizia a seguire una software house

```
CREATE OR REPLACE FUNCTION Segui_Software_House(
      u_nick varchar,
      sh_piva varchar(20))
RETURNS void AS $$
DECLARE
      numseguiti integer;
BEGIN
IF EXISTS(SELECT 1 FROM SoftwareHouse WHERE sh_piva = PartitaIVA)
AND EXISTS(SELECT 1 FROM Utente WHERE u_nick = Nickname) THEN
      IF NOT EXISTS(SELECT 1 FROM SeguitoAttuale WHERE sh_piva =
      SoftwareHouse AND u_nick = Utente) THEN
            INSERT INTO SeguitoAttuale(Utente, SoftwareHouse,
            InizioSeguito)
            VALUES (u_nick, sh_piva, CURRENT_DATE);

            UPDATE SoftwareHouse
            SET NumeroSeguaci = NumeroSeguaci + 1
            WHERE PartitaIVA = sh_piva;
```

```
            IF(EXISTS(SELECT 1 FROM SeguitoPassato WHERE u_nick =
            Utente AND sh_piva = SoftwareHouse)) THEN
                  DELETE FROM SeguitoPassato
                  WHERE u_nick = Utente AND sh_piva = SoftwareHouse;
            END IF;
            RAISE NOTICE 'Operazione completata: % ora segue %',
            u_nick, sh_piva;
      ELSE
            RAISE NOTICE 'L''utente % segue già questa Software
            House.', u_nick;
      END IF;
ELSE
      RAISE NOTICE 'Errore: Utente e/o Software House non
      trovati.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP15: Utente smette di seguire una software house

```
CREATE OR REPLACE FUNCTION Smetti_Seguire_SoftwareHouse(
      u_nick varchar,
      sh_piva varchar(20)
)
RETURNS void AS $$
DECLARE
      numseguiti integer;
      inizio date;
BEGIN
IF EXISTS(SELECT 1 FROM SeguitoAttuale WHERE u_nick = Utente AND
sh_piva = SoftwareHouse) THEN
      inizio := (SELECT InizioSeguito FROM SeguitoAttuale WHERE
      sh_piva = SoftwareHouse AND u_nick = Utente);
      INSERT INTO SeguitoPassato(Utente, SoftwareHouse,
      InizioSeguito, FineSeguito) VALUES (u_nick, sh_piva, inizio,
      CURRENT_DATE)
      ON CONFLICT (Utente, SoftwareHouse) DO UPDATE
```

```
      SET InizioSeguito = EXCLUDED.InizioSeguito, FineSeguito =
      EXCLUDED.FineSeguito;

      DELETE FROM SeguitoAttuale
      WHERE Utente = u_nick AND sh_piva = SoftwareHouse;
      UPDATE SoftwareHouse
      SET NumeroSeguaci = NumeroSeguaci - 1
      WHERE PartitaIVA = sh_piva;
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP16: Ricerca degli acquisti effettuati in una certa data da parte di un utente, segnalando la spesa totale per ciascun acquisto

```
CREATE OR REPLACE FUNCTION Ricerca_Acquisti(
      u_nick varchar,
      a_data date
)
RETURNS TABLE(Codice_Acquisto integer, id_MetodoPagamento integer,
Prezzo_totale decimal) AS $$
BEGIN
IF EXISTS (SELECT 1 FROM Utente WHERE Nickname=u_nick) THEN
      RETURN QUERY
      SELECT IdAcquisto, MetodoPagamento, sum(PrezzoAcquisto)
      FROM Acquisto, Composizione
      WHERE Data=a_data AND Utente=u_nick AND Acquisto=IdAcquisto
      GROUP BY IdAcquisto, MetodoPagamento;
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP17: Ricarica del saldo di un utente

```
CREATE OR REPLACE FUNCTION Aggiorna_Saldo (
      aggiunta_saldo decimal,
      u_nick varchar
)
```

```
RETURNS void AS $$
BEGIN
IF EXISTS (SELECT 1 FROM Utente WHERE Nickname=u_nick) AND
aggiunta_saldo > 0 THEN
      UPDATE MetodoDiPagamento
      SET Disponibilita = Disponibilita + aggiunta_saldo
      WHERE Proprietario = u_nick AND Tipo = 'Saldo';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP18: Visualizzare in ordine alfabetico i contenuti all'interno della libreria di un utente

```
CREATE OR REPLACE FUNCTION Visualizza_libreria (
      u_nick varchar
)
RETURNS TABLE(NomeGioco varchar, OreGioco time, UltimaDataAvvio
date, Preferito boolean) AS $$
BEGIN
IF EXISTS (SELECT 1 FROM Utente WHERE Nickname=u_nick) THEN
      RETURN QUERY
      SELECT Nome, OreDiGioco, UltimaDataAvvio, Preferito
      FROM ContenutoVideoludico C, Libreria L
      WHERE C.IdContenuto=L.Contenuto AND L.Utente=u_nick
      ORDER BY Nome;
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP19: Verifica che un utente possiede l'età minima per un certo videogioco

```
CREATE OR REPLACE FUNCTION Verifica_eta (
      u_nick varchar,
      v_id integer
)
RETURNS boolean AS $$
DECLARE
      data_nascita date;
      pegi integer;
```

```
      eta_utente integer;
BEGIN
IF EXISTS (SELECT 1 FROM Utente WHERE Nickname=u_nick) AND EXISTS
(SELECT 1 FROM Videogioco WHERE IdVideogioco=v_id) THEN
      SELECT DataDiNascita INTO data_nascita
      FROM Utente
      WHERE Nickname=u_nick;

      SELECT PEGI INTO pegi
      FROM Videogioco
      WHERE IdVideogioco=v_id;

      eta_utente := EXTRACT(YEAR FROM AGE(CURRENT_DATE,
      data_nascita));

      IF (eta_utente >= pegi) THEN
            RAISE NOTICE 'Utente idoneo al contenuto.';
            RETURN TRUE;
      ELSE
            RAISE NOTICE 'Utente non idoneo al contenuto.';
            RETURN FALSE;
      END IF;
ELSE
      RAISE NOTICE 'Utente e/o videogioco non trovati.';
      RETURN FALSE;
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP20: Visualizzazione delle espansioni acquistabili di un videogioco

```
CREATE OR REPLACE FUNCTION Visualizza_espansioni(
      v_id integer
)
RETURNS TABLE(NomeEspansione varchar, DimensioneEspansione
decimal, PrezzoDiBase decimal, PrezzoDiVendita decimal) AS $$
BEGIN
IF EXISTS (SELECT 1 FROM Videogioco WHERE IdVideogioco=v_id) THEN
```

```
      RETURN QUERY
      SELECT Nome, Dimensione, PrezzoBase, PrezzoVendita
      FROM ContenutoVideoludico
      WHERE EstensioneDi=v_id AND Tipo='Espansione';

      IF NOT FOUND THEN RAISE NOTICE 'Nessuna espansione trovata
      per il videogioco con ID %.', v_id;
      END IF;
ELSE
      RAISE NOTICE 'Errore: videogioco non individuato.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP21: Ricerca di contenuti filtrata da prezzo massimo, genere, data di uscita minima, software house editrice

```
CREATE OR REPLACE FUNCTION Ricerca_Filtrata(
      f_prezzo_max decimal,
      f_genere varchar,
      f_datauscita date,
      f_piva_softwarehouse varchar(20)
)
RETURNS TABLE (Nome varchar, Prezzo decimal, DataDiUscita date,
SoftwareHouse varchar) AS $$
BEGIN
      RETURN QUERY
      SELECT C.Nome, C.PrezzoVendita, P.DataRilascio, S.Nome
      FROM ContenutoVideoludico C, Pubblicazione P, SoftwareHouse
      S, Appartenenza A
      WHERE C.PrezzoVendita<=f_prezzo_max AND
      C.IdContenuto=A.Contenuto AND A.Genere=f_genere AND
      P.DataRilascio>=f_datauscita AND P.Contenuto=C.IdContenuto
      AND P.SoftwareHouse=S.PartitaIVA AND
      S.PartitaIVA=f_piva_softwarehouse;
END;
$$ LANGUAGE plpgsql;
```

OP22: Visualizzazione degli obiettivi di un videogioco

```
CREATE OR REPLACE FUNCTION Visualizza_obiettivi (
      v_id integer
)
RETURNS TABLE(Numero integer, Nome varchar, Descrizione varchar)
AS $$
BEGIN
IF EXISTS (SELECT 1 FROM Videogioco WHERE IdVideogioco=v_id) THEN
      RETURN QUERY
      SELECT O.Numero, O.Nome, O.Descrizione
      FROM Obiettivo O
      WHERE O.Videogioco=v_id
      ORDER BY O.Numero;

      IF NOT FOUND THEN RAISE NOTICE 'Nessun obiettivo registrato
      per il videogioco con ID %.', v_id;
      END IF;
ELSE
      RAISE NOTICE 'Errore: nessun videogioco individuato.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP23: Consultazione delle specifiche hardware minime richieste per un videogioco

```
CREATE OR REPLACE FUNCTION Specifiche_Hardware_Minime(
      v_id integer
      )
RETURNS TABLE(TipoComponente varchar, NomeModello varchar) AS $$
BEGIN
IF EXISTS(SELECT 1 FROM Videogioco WHERE IdVideogioco=v_id) THEN
      RETURN QUERY
      SELECT H.Tipo, H.Nome
      FROM Hardware H, Minimo M
      WHERE H.IdHardware=M.Hardware AND M.Videogioco=v_id;
```

```
      IF NOT FOUND THEN
            RAISE NOTICE 'Il videogioco non ha specifiche minime
            registrate.';
      END IF;
ELSE
      RAISE NOTICE 'Errore: videogioco non individuato.';
END IF;
END;
$$ LANGUAGE plpgsql
```

OP24: Consultazione delle specifiche hardware consigliate richieste per un videogioco

```
CREATE OR REPLACE FUNCTION Specifiche_Hardware_Consigliate(
      v_id integer
      )
RETURNS TABLE(TipoComponente varchar, NomeModello varchar) AS $$
BEGIN
IF EXISTS(SELECT 1 FROM Videogioco WHERE IdVideogioco=v_id) THEN
      RETURN QUERY
      SELECT H.Tipo, H.Nome
      FROM Hardware H, Consigliato C
      WHERE H.IdHardware=C.Hardware AND C.Videogioco=v_id;

      IF NOT FOUND THEN
            RAISE NOTICE 'Il videogioco non ha specifiche
            consigliate registrate.';
      END IF;
ELSE
      RAISE NOTICE 'Errore: videogioco non individuato.';
END IF;
END;
$$ LANGUAGE plpgsql
```

OP25: Consultazione dei sistemi operativi supportati da un videogioco

```
CREATE OR REPLACE FUNCTION Sistemi_Operativi_Compatibili(
```

```
      v_id integer
      )
RETURNS TABLE(NomeSO varchar, VersioneSO varchar, ArchitetturaSO
varchar, ProduttoreSO varchar) AS $$
BEGIN
IF EXISTS (SELECT 1 FROM Videogioco WHERE IdVideogioco=v_id) THEN
      RETURN QUERY
      SELECT SO.Nome, SO.Versione, SO.Architettura, SO.Produttore
      FROM SistemaOperativo SO, Compatibilita C
      WHERE SO.IdSistemaOperativo=C.SistemaOperativo AND
      C.Videogioco=v_id;

      IF NOT FOUND THEN
      RAISE NOTICE 'Nessun sistema operativo individuato.';
      RETURN NULL;
      END IF;
ELSE
      RAISE EXCEPTION 'Errore: videogioco non individuato.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP26: Visualizzazione delle recensioni di un videogioco

```
CREATE OR REPLACE FUNCTION Visualizza_Recensioni(
      v_id integer
)
RETURNS TABLE(Utente varchar, Valutazione decimal, Commento
varchar) AS $$
BEGIN
IF EXISTS (SELECT 1 FROM Videogioco WHERE IdVideogioco=v_id) THEN
      RETURN QUERY
      SELECT Utente, Stelle, COALESCE(Commento, 'Nessun commento.')
      FROM Recensione
      WHERE v_id = Videogioco;

      IF NOT FOUND THEN
```

```
            RAISE NOTICE 'Nessuna recensione registrata.';
      END IF;
ELSE
      RAISE NOTICE 'Errore: videogioco non individuato.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP27: Aggiornamento del prezzo di vendita

```
CREATE OR REPLACE FUNCTION Aggiorna_PrezzoVendita(
      c_id integer,
      nuovo_prezzo_vendita decimal
)
RETURNS void AS $$
DECLARE
      prezzo_base decimal;
BEGIN
IF EXISTS (SELECT 1 FROM ContenutoVideoludico WHERE
IdContenuto=c_id) THEN
      SELECT PrezzoBase INTO prezzo_base
      FROM ContenutoVideoludico
      WHERE IdContenuto=c_id;

      IF nuovo_prezzo_vendita>0 AND
      nuovo_prezzo_vendita<=prezzo_base THEN
            UPDATE ContenutoVideoludico
            SET PrezzoVendita=nuovo_prezzo_vendita
            WHERE IdContenuto=c_id;

            RAISE NOTICE 'Aggiornato prezzo di vendita del contenuto
            con ID %.', c_id;
      ELSE
            RAISE NOTICE 'Nuovo prezzo di vendita non idoneo.';
      END IF;
ELSE
      RAISE NOTICE 'Errore: contenuto non individuato.';
```

```
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP28: Visualizzare la valutazione media in stelle di un videogioco

```
CREATE OR REPLACE FUNCTION Valutazione_Media_Videogioco(
      v_id integer
)
RETURNS decimal AS $$
DECLARE
      media decimal;
BEGIN
IF EXISTS (SELECT 1 FROM Videogioco WHERE IdVideogioco=v_id) THEN
      SELECT ROUND(AVG(Stelle), 1) INTO media
      FROM Recensione
      WHERE Videogioco = v_id;

      media:=COALESCE(media, 0);

      UPDATE Videogioco
      SET MediaStelle = media
      WHERE v_id = IdVideogioco;

      RETURN media;
ELSE
      RAISE EXCEPTION 'Errore: videogioco non individuato.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP29: Aggiunta o aggiornamento di una recensione da parte di un utente

```
CREATE OR REPLACE FUNCTION Crea_o_Aggiorna_Recensione(
      v_id integer,
      u_nick varchar,
      stelle decimal,
```

```
      commento varchar
)
RETURNS void AS $$
BEGIN
IF EXISTS(SELECT 1 FROM ContenutoVideoludico WHERE v_id =
IdContenuto) AND EXISTS(SELECT 1 FROM Utente WHERE u_nick =
Nickname) AND EXISTS(SELECT 1 FROM Libreria WHERE Utente=u_nick
AND Contenuto=v_id) THEN
      IF EXISTS (SELECT 1 FROM Recensione WHERE u_nick = Utente AND
      v_id = Videogioco) THEN
            UPDATE Recensione
            SET Stelle = stelle, Commento = commento
            WHERE u_nick = Utente AND v_id = Videogioco;
      ELSE
            INSERT INTO Recensione(Utente, Videogioco, Stelle,
            Commento)
            VALUES (u_nick, v_id, stelle, commento);
      END IF;
PERFORM Valutazione_Media_Videogioco(v_id);
ELSE
      RAISE NOTICE 'Errore: l'utente non possiede il videogioco,
      oppure l'utente e/o il videogioco non sono validi.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP30: Visualizzare le Software house che un utente segue attualmente

```
CREATE OR REPLACE FUNCTION Software_House_Seguite(
      u_nick varchar
)
RETURNS TABLE(SoftwareHouseSeguite varchar)AS $$
BEGIN
IF EXISTS (SELECT 1 FROM Utente WHERE Nickname=u_nick) THEN
      RETURN QUERY
      SELECT SH.Nome
      FROM SeguitoAttuale SA, SoftwareHouse SH, Utente U
```

```
      WHERE SA.Utente=U.Nickname AND SA.SoftwareHouse=SH.PartitaIVA
      AND U.Nickname=u_nick;

      IF NOT FOUND THEN THEN
            RAISE NOTICE 'L'utente non segue alcuna software
      house.';
      END IF;
ELSE
      RAISE NOTICE 'Errore: utente non individuato.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP31: Visualizzare il numero di seguaci di una Software house

```
CREATE OR REPLACE FUNCTION Numero_Seguaci(
      sh_piva varchar
)
RETURNS integer AS $$
DECLARE
      num_seguaci integer;
BEGIN
num_seguaci := (
      SELECT NumeroSeguaci
      FROM SoftwareHouse
      WHERE sh_piva = PartitaIVA);
RETURN num_seguaci;
END;
$$ LANGUAGE plpgsql;
```

OP32: Aggiunta di un server da parte di una software house

```
CREATE OR REPLACE FUNCTION Aggiunta_Server(
      s_ip varchar,
      s_nome varchar,
```

```
      s_tipo varchar,
      s_numses integer,
      sh_piva varchar(20)
)
RETURNS void AS $$
BEGIN
IF(EXISTS(SELECT 1 FROM SoftwareHouse WHERE sh_piva = PartitaIVA)
AND NOT EXISTS(SELECT 1 FROM Server WHERE s_ip = IP)) THEN
      INSERT INTO Server(IP, Nome,Tipo, Stato, LimiteSessioni,
      Amministratore)
      VALUES (s_ip, s_nome, s_tipo, 'Aperto', s_numses, sh_piva);
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP33: Chiusura di un server da parte di una software house

```
CREATE OR REPLACE FUNCTION Chiusura_Server(
      s_ip varchar,
      sh_piva varchar(20)
)
RETURNS void AS $$
BEGIN
IF(EXISTS(SELECT 1 FROM Server WHERE s_ip = IP AND sh_piva =
Amministratore)) THEN
      DELETE FROM Sessione
      WHERE Server = s_ip;

      UPDATE Server
      SET Stato = 'Chiuso'
      WHERE s_ip = IP;
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP34: Apertura di un community server (cambiamento di stato) da parte di un utente

```
CREATE OR REPLACE FUNCTION Apertura_Community_Server(
      u_nick varchar,
      s_ip varchar
)
RETURNS void AS $$
BEGIN
IF(EXISTS(SELECT 1 FROM Gestione WHERE u_nick = Utente AND s_ip =
Server)) THEN
      UPDATE Server
      SET Stato = 'Aperto'
      WHERE IP = s_ip AND Stato = 'Chiuso';
      RAISE NOTICE 'Il server % è aperto', s_ip;
ELSE
      RAISE EXCEPTION 'Errore: impossibile aprire il server %',
      s_ip;
END IF;
END;
$$ LANGUAGE plpgsql;
```

OP35: Aggiunta di una software house

```
CREATE OR REPLACE FUNCTION Aggiunta_Software_House(
      sh_piva varchar(20),
      sh_nome varchar,
      sh_citta varchar
)
RETURNS void AS $$
BEGIN
sh_piva := UPPER(TRIM(sh_piva));
IF(NOT EXISTS(SELECT 1 FROM SoftwareHouse WHERE sh_piva =
PartitaIVA)) THEN
      INSERT INTO SoftwareHouse(PartitaIVA, Nome, Citta)
      VALUES (sh_piva, sh_nome, sh_citta);
ELSE
      RAISE EXCEPTION 'Errore: esiste già una software house
registrata con la Partita IVA %', sh_piva;
END IF;
```

```
END;
$$ LANGUAGE plpgsql;
```

OP36: Rimozione di una software house

```
CREATE OR REPLACE FUNCTION Rimozione_Software_House(
      sh_piva varchar(20)
)
RETURNS void AS $$
DECLARE
      sh_nome varchar;
BEGIN
sh_piva := UPPER(TRIM(sh_piva));
IF(EXISTS(SELECT 1 FROM SoftwareHouse WHERE sh_piva = PartitaIVA))
THEN
      sh_nome := (
      SELECT Nome
      FROM SoftwareHouse
      WHERE PartitaIVA = sh_piva);
      DELETE FROM SoftwareHouse
      WHERE sh_piva = PartitaIVA;
      RAISE NOTICE 'La software house % è stata eliminata',
      sh_nome;
ELSE
      RAISE EXCEPTION 'Errore: la software house non è stata
eliminata';
END IF;
END;
$$ LANGUAGE plpgsql;
```

Esecuzione delle operazioni principali

OP1: Registrazione di un utente

Vediamo in che modo agisce la seguente operazione nelle tabelle del database.

Supponiamo di avere la seguente istanza per la relazione UTENTE:

	nickname [PK] character varying (20)	telefono character varying (15)	email character varying (50)
1	AaraoSilva	+551191234-5678	asilva@uol.com.br
2	ChloeWang	+8613912345678	chloe.wang@qq.com
3	EdwardCullen	+1(555)270-2488	edward.cullen@gmail.co…
4	ElenaGomez	+34600000000	elena.g@hotmail.es
5	FatimaAli	+971501234567	f.ali@emirates.ae
6	HansMuller	+4915123456789	h.muller@t-online.de
7	JeanDupont	+33612345678	j.dupont@orange.fr
8	Joseh42	3516577839	joseph.brown@gmail.com
9	LukasNovak	+420720123456	lukas.novak@seznam.cz
10	MarcoRossi88	+39 347 1234567	m.rossi88@gmail.com
11	MarioRossi_	3315672829	mario.rossi@gmail.com
12	SarahSmith	+1(202)555-0143	sarah.smith@outlook.com
13	Valentina124	3240757507	vale.miao@gmail.com
14	YukiTanaka	+8190-1234-5678	yuki.t@yahoo.jp

password character varying (50)	nazionalita character varying (30)	livello integer	datadinascita date	datadiiscrizione date
RioDeJaneiro	Brasiliana	1	1989-07-19	2026-01-08
Shanghai2022	Cinese	1	1998-02-14	2026-01-08
IloveBella	Americana	1	2010-01-01	2026-01-08
Barcelona99	Spagnola	1	2000-12-25	2026-01-08
DesertSun21	Emira	1	1996-09-05	2026-01-08
BerlinWall89	Tedesca	1	1978-01-10	2026-01-08
AllezLesBleus	Francese	1	1985-08-12	2026-01-08
jos42uh	Inglese	1	2000-01-09	2026-01-08
Praha_Rules	Ceca	1	1993-06-30	2026-01-08
Password123!	Italiana	1	1988-05-15	2026-01-08
mar10r0ss1	Svizzera	1	1982-10-31	2026-01-08
S3curePass!	Americana	1	1995-11-22	2026-01-08
valemiao	Italiana	1	2003-12-05	2026-01-08
Tokyo2024	Giapponese	1	1992-03-03	2026-01-08

Supponiamo di voler registrare un nuovo utente tramite la funzione:

```
SELECT Registra_Utente ('Paolo696', '+39 3391148487',
'paolorossi@gmail.com', 'paolo1234', 'Italiana', '12/02/1997')
```

La relazione UTENTE viene aggiornata con i dati del nuovo utente registrato.

	nickname [PK] character varying (20)	telefono character varying (15)	email character varying (50)
1	AaraoSilva	+551191234-5678	asilva@uol.com.br
2	ChloeWang	+8613912345678	chloe.wang@qq.com
3	EdwardCullen	+1(555)270-2488	edward.cullen@gmail.co...
4	ElenaGomez	+34600000000	elena.g@hotmail.es
5	FatimaAli	+971501234567	f.ali@emirates.ae
6	HansMuller	+4915123456789	h.muller@t-online.de
7	JeanDupont	+33612345678	j.dupont@orange.fr
8	Joseh42	+39 3516577839	joseph.brown@gmail.com
9	LukasNovak	+420720123456	lukas.novak@seznam.cz
10	MarcoRossi88	+39 347 1234567	m.rossi88@gmail.com
11	MarioRossi_	+39 3315672829	mario.rossi@gmail.com
12	Paolo696	+39 3391148487	paolorossi@gmail.com
13	SarahSmith	+1(202)555-0143	sarah.smith@outlook.com
14	Valentina124	+39 3240757507	vale.miao@gmail.com
15	YukiTanaka	+8190-1234-5678	yuki.t@yahoo.jp

password character varying (50)	nazionalita character varying (30)	livello integer	datadinascita date	datadiiscrizione date
RioDeJaneiro	Brasiliana	1	1989-07-19	2026-01-08
Shanghai2022	Cinese	1	1998-02-14	2026-01-08
IloveBella	Americana	1	2010-01-01	2026-01-08
Barcelona99	Spagnola	1	2000-12-25	2026-01-08
DesertSun21	Emira	1	1996-09-05	2026-01-08
BerlinWall89	Tedesca	1	1978-01-10	2026-01-08
AllezLesBleus	Francese	1	1985-08-12	2026-01-08
jos42uh	Inglese	1	2000-01-09	2026-01-08
Praha_Rules	Ceca	1	1993-06-30	2026-01-08
Password123!	Italiana	1	1988-05-15	2026-01-08
mar10r0ss1	Svizzera	1	1982-10-31	2026-01-08
paolo1234	Italiana	1	1997-02-12	2026-01-08
S3curePass!	Americana	1	1995-11-22	2026-01-08
valemiao	Italiana	1	2003-12-05	2026-01-08
Tokyo2024	Giapponese	1	1992-03-03	2026-01-08

Inoltre l'aggiunta di un utente provoca anche la creazione automatica del METODO DI PAGAMENTO 'Saldo' con Disponibilità=0 fino a quando l'utente non decide di ricaricarlo.

	idmetodo [PK] integer	disponibilita numeric (10,2)	tipo character varying (20)	iban character varying (11)	intestatario character varying (50)	proprietario character varying (20)
1	1	0.00	Saldo	[null]	[null]	MarcoRossi88
2	2	0.00	Saldo	[null]	[null]	SarahSmith
3	3	0.00	Saldo	[null]	[null]	YukiTanaka
4	4	0.00	Saldo	[null]	[null]	JeanDupont
5	5	0.00	Saldo	[null]	[null]	ElenaGomez
6	6	0.00	Saldo	[null]	[null]	HansMuller
7	7	0.00	Saldo	[null]	[null]	ChloeWang
8	8	0.00	Saldo	[null]	[null]	LukasNovak
9	9	0.00	Saldo	[null]	[null]	FatimaAli
10	10	0.00	Saldo	[null]	[null]	AaraoSilva
11	11	0.00	Saldo	[null]	[null]	EdwardCullen
12	12	0.00	Saldo	[null]	[null]	Valentina1234

METODO DI PAGAMENTO prima dell'aggiunta dell'utente

	idmetodo [PK] integer	disponibilita numeric (10,2)	tipo character varying (20)	iban character varying (11)	intestatario character varying (50)	proprietario character varying (20)
1	1	0.00	Saldo	[null]	[null]	MarcoRossi88
2	2	0.00	Saldo	[null]	[null]	SarahSmith
3	3	0.00	Saldo	[null]	[null]	YukiTanaka
4	4	0.00	Saldo	[null]	[null]	JeanDupont
5	5	0.00	Saldo	[null]	[null]	ElenaGomez
6	6	0.00	Saldo	[null]	[null]	HansMuller
7	7	0.00	Saldo	[null]	[null]	ChloeWang
8	8	0.00	Saldo	[null]	[null]	LukasNovak
9	9	0.00	Saldo	[null]	[null]	FatimaAli
10	10	0.00	Saldo	[null]	[null]	AaraoSilva
11	11	0.00	Saldo	[null]	[null]	EdwardCullen
12	12	0.00	Saldo	[null]	[null]	Valentina1234
13	13	0.00	Saldo	[null]	[null]	Paolo696

METODO DI PAGAMENTO dopo l'aggiunta dell'utente

Si noti che i campi IBAN e Intestatario sono NULL in quanto l'utente appena registrato possiede solo il metodo di pagamento di tipo 'Saldo'.

OP8: Visualizzazione del profilo utente

Per visualizzare il profilo dell'utente utilizziamo la funzione seguente:

```
SELECT * from visualizza_dati_utente('Paolo696')
```

L'output della funzione è il seguente:

	nicknameutente character varying	nazionalitautente character varying	livelloutente integer	datadiiscrizioneutente date
1	Paolo696	Italiana	1	2026-01-08

OP10: Pubblicazione di un contenuto

Supponiamo di voler pubblicare un videogioco sulla piattaforma, utilizziamo la funzione:

```
SELECT Gestione_Pubblicazione(
c_titolo:='CyberPunk2077',
c_dimensione:=70.5,
c_prezzobase:=59.99,
c_tipo:='Videogioco',
sh_sviluppo:=ARRAY['53972120595', '40599270895'],
sh_pubblicazione:=ARRAY['53972120595'],
c_generi:=ARRAY['RPG', 'Open World', 'Sci-fi'],

v_pegi:=18,
v_tipo_gioco:='Singleplayer',
v_so_id:=ARRAY[1, 2],
v_hw_minimo:=ARRAY[1, 2, 3],
v_hw_consigliato:=ARRAY[4, 5, 6]
)
```

La relazione CONTENUTO VIDEOLUDICO viene aggiornata in questo modo:

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1

	idcontenuto [PK] integer	nome character varying (50)	dimensione numeric (6,2)	prezzobase numeric (10,2)	prezzovendita numeric (10,2)	tipo character varying (20)	estensionedi integer	relativaa integer
1	1	CyberPunk2077	70.50	59.99	59.99	Videogioco	[null]	[null]

Inoltre il contenuto viene aggiunto anche nella relazione PUBBLICAZIONE, insieme alla/e P.IVA della/e Software House che ha/hanno pubblicato il contenuto.

	contenuto [PK] integer	softwarehouse [PK] character (11)	datarilascio date
1	1	53972120595	2026-01-08

Analogamente il contenuto viene inserito anche nella relazione SVILUPPO, insieme alla/e P.IVA della/e Software House che ha/hanno sviluppato il contenuto.

	contenuto [PK] integer	softwarehouse [PK] character (11)
1	1	40599270895
2	1	53972120595

La relazione SOFTWARE HOUSE è la seguente:

	partitaiva [PK] character (11)	nome character varying (50)	citta character varying (50)	numeroseguaci integer
1	12345678901	FromSoftware	Tokyo	300
2	40599270895	SAP SE	Walldorf	2000
3	53972120595	Microsoft Corporation	Redmond	1800
4	98765432109	Bandai Namco Entertainment	Roma	560

Il contenuto pubblicato possiede uno o più generi, perciò lo inseriamo anche nella relazione APPARTENENZA.

	contenuto [PK] integer	genere [PK] character varying (30)
1	1	Open World
2	1	RPG
3	1	Sci-fi

Il contenuto si tratta di un videogioco, di conseguenza andrà inserito anche nella relazione VIDEOGIOCO.

	idvideogioco [PK] integer	tipo character varying (20)	pegi integer	mediastelle numeric (2,1)
1	1	Singleplayer	18	0.0

Ogni videogioco possiede dei requisiti Hardware minimo e consigliato e anche dei sistemi operativi compatibili; lo andiamo a inserire nelle relazioni MINIMO, CONSIGLIATO e COMPATIBILITA', le modifiche sono mostrate di seguito:

	videogioco [PK] integer	hardware [PK] integer
1	1	1
2	1	2
3	1	3

Relazione MINIMO

	videogioco [PK] integer	hardware [PK] integer
1	1	4
2	1	5
3	1	6

Relazione CONSIGLIATO

	videogioco [PK] integer	sistemaoperativo [PK] integer
1	1	1
2	1	2

Relazione COMPATIBILITA’

Di seguito anche le relazioni HARDWARE e SISTEMA OPERATIVO:

	idhardware [PK] integer	tipo character varying (20)	nome character varying (20)
1	1	GPU	GeForce GT 1030
2	2	CPU	Intel Core i9
3	3	RAM	Corsair LPX
4	4	GPU	GeForce RTX 400
5	5	CPU	AMD Ryzen
6	6	RAM	Trident Z5 Neo
7	7	CPU	Ryzen 7800X3D
8	8	RAM	16GB DDR5

Relazione HARDWARE

	idsistemaoperativo [PK] integer	nome character varying (20)	versione character varying (20)	produttore character varying (20)	architettura character varying (10)
1	1	Windows	10	Microsoft	x86
2	2	Windows	11	Microsoft	x64
3	3	Linux	Ubuntu 22.04 LTS	Canonical	x64
4	4	Linux	SteamOS 3.5	Valve Corporation	x64

Relazione SISTEMA OPERATIVO

OP11: Rimozione di un contenuto

Supponiamo di voler rimuovere il contenuto ‘Elden Ring’ dalla piattaforma, utilizziamo la funzione:

```
SELECT Rimozione_Contenuto_Videoludico(id:=2)
```

Si riporta la relazione CONTENUTO VIDEOLUDICO prima e dopo la rimozione.

Showing rows: 1 to 2 Page No: 1

	idcontenuto [PK] integer	nome character varying (50)	dimensione numeric (6,2)	prezzobase numeric (10,2)	prezzovendita numeric (10,2)	tipo character varying (20)	estensionedi integer	relativaa integer
1	1	CyberPunk2077	70.50	59.99	59.99	Videogioco	[null]	[null]
2	2	Elden Ring	60.00	59.99	59.99	Videogioco	[null]	[null]

	idcontenuto [PK] integer	nome character varying (50)	dimensione numeric (6,2)	prezzobase numeric (10,2)	prezzovendita numeric (10,2)	tipo character varying (20)	estensionedi integer	relativaa integer
1	1	CyberPunk2077	70.50	59.99	59.99	Videogioco	[null]	[null]

Si nota che, una volta eliminata la tupla dal contenuto videoludico, questa verrà eliminata da tutte le relazioni in cui è presente la sua chiave.
Vediamo la relazione VIDEOGIOCO prima e dopo la rimozione.

	idvideogioco [PK] integer	tipo character varying (20)	pegi integer	mediastelle numeric (2,1)
1	1	Singleplayer	18	0.0
2	2	Multiplayer	16	0.0

	idvideogioco [PK] integer	tipo character varying (20)	pegi integer	mediastelle numeric (2,1)
1	1	Singleplayer	18	0.0

Vediamo la relazione COMPATIBILITA’:

	videogioco [PK] integer	sistemaoperativo [PK] integer
1	1	1
2	1	2
3	2	1
4	2	3

	videogioco [PK] integer	sistemaoperativo [PK] integer
1	1	1
2	1	2

La relazione APPARTENENZA:

	contenuto [PK] integer	genere [PK] character varying (30)
1	1	Open World
2	1	RPG
3	1	Sci-fi
4	2	Fantasy
5	2	Open World
6	2	RPG

	contenuto [PK] integer	genere [PK] character varying (30)
1	1	Open World
2	1	RPG
3	1	Sci-fi

La relazione CONSIGLIATO:

	videogioco [PK] integer	hardware [PK] integer
1	1	4
2	1	5
3	1	6
4	2	1
5	2	4
6	2	8

	videogioco [PK] integer	hardware [PK] integer
1	1	4
2	1	5
3	1	6

La relazione MINIMO:

	videogioco [PK] integer	hardware [PK] integer
1	1	1
2	1	2
3	1	3
4	2	2
5	2	7

	videogioco [PK] integer	hardware [PK] integer
1	1	1
2	1	2
3	1	3

La relazione SVILUPPO:

	contenuto [PK] integer	softwarehouse [PK] character (11)
1	1	40599270895
2	1	53972120595
3	2	12345678901

	contenuto [PK] integer	softwarehouse [PK] character (11)
1	1	40599270895
2	1	53972120595

E infine la relazione PUBBLICAZIONE:

	contenuto [PK] integer	softwarehouse [PK] character (11)	datarilascio date
1	1	53972120595	2026-01-08
2	2	98765432109	2026-01-08

	contenuto [PK] integer	softwarehouse [PK] character (11)	datarilascio date
1	1	53972120595	2026-01-08

OP21: Ricerca di contenuti filtrata da prezzo massimo, genere, data di uscita minima, software house editrice

Per eseguire questa operazione, utilizziamo la seguente funzione:

```
SELECT * FROM Ricerca_Filtrata (60.0, 'Fantasy', CURRENT_DATE,
'98765432109')
```

L'output della funzione è il seguente:

	nome character varying	prezzo numeric	datadiuscita date	softwarehouse character varying
1	CyberPunk2077	59.99	2026-01-08	Microsoft Corporation

Oppure se eseguiamo:

```
SELECT * FROM Ricerca_Filtrata (60.0, 'RPG', CURRENT_DATE,
'53972120595')
```

L'output sarà:

	nome character varying	prezzo numeric	datadiuscita date	softwarehouse character varying
1	Elden Ring	59.99	2026-01-08	Bandai Namco Entertainment

OP24: Consultazione delle specifiche hardware consigliate richieste per un videogioco

Vogliamo riportare le specifiche relative al videogioco 'Cyberpunk2077':

```
SELECT * FROM Specifiche_Hardware_Consigliate(v_id:=1)
```

L'output della funzione è il seguente:

	tipocomponente character varying	nomemodello character varying
1	GPU	GeForce RTX 400
2	CPU	AMD Ryzen
3	RAM	Trident Z5 Neo

Vogliamo visualizzare le specifiche relative al videogioco ‘Elden Ring’:

```
SELECT * FROM Specifiche_Hardware_Consigliate(v_id:=2)
```

L’output sarà:

	tipocomponente character varying	nomemodello character varying
1	GPU	GeForce GT 1030
2	GPU	GeForce RTX 400
3	RAM	16GB DDR5